

CRITICAL LITERATURE REVIEW

Agentic Computer-Aided Engineering, Finite-Element Automation and Scientific Machine Learning

*A full-text synthesis of models, methods, code architectures,
evidence, novelties, limitations and open research gaps*

20 works • 555 source pages • corpus dated 2023–2026

Scope: full-paper review of the uploaded PDFs, including mathematical formulations, orchestration logic, retrieval and memory, code-generation and execution loops, benchmark construction, quantitative results, ablations, implementation details, reproducibility, and claim-to-evidence alignment.

Prepared 19 August 2026

Executive synthesis

This review analyzes the complete set of **20 uploaded works** rather than an abstract-only subset. The corpus contains 555 PDF pages and spans 2023–2026. It is dominated by large-language-model systems for finite-element analysis, computational fluid dynamics, computer-aided engineering, and scientific machine learning, but it also contains three important boundary cases: a field-level scoping review, a repository-level software benchmark whose acronym “FEA” does **not** mean finite-element analysis, and a theoretical gravitation paper unrelated to agentic engineering software. The corpus is therefore best understood as evidence about four linked but distinct questions: what LLMs can generate; what tool-using agents can execute; what scientific claims their outputs justify; and how those claims are currently evaluated.

The clearest technical trend is a move from monolithic prompting toward **typed, stateful, tool-mediated workflows**. The stronger systems do not merely ask an LLM to “write a solver.” They decompose the workflow into problem formalization, geometry or mesh generation, input-deck or code construction, execution, error diagnosis, result inspection, and persistent memory. Examples include Foam-Agent 2.0’s six-agent OpenFOAM graph; FeaGPT’s planner–FreeCAD–Gmsh–CalculiX–analysis chain; VFEAgent’s visual extraction followed by Text2Abaqus generation; PDE-Agents’ LangGraph supervisor coupled to DOLFINx and a knowledge graph; and ATHENA’s hierarchical separation between a first-order implementation operator and a second-order scientific-strategy operator. This architectural separation is a real advance because it exposes intermediate artifacts that can be inspected and repaired.

However, **workflow completion is repeatedly conflated with simulation correctness**. Several papers define success as executable code, a completed solver run, the presence of required files, or qualitative resemblance of plots. Those are necessary conditions, but they do not establish the correctness of the weak form, element formulation, constitutive law, boundary tags, mesh adequacy, units, convergence, solver tolerances, or quantity of interest. Tian et al. provide the strongest direct counterexample: configurations that sometimes generate executable or visually plausible FEniCS solutions nevertheless produce no physically valid solution on the two constitutively demanding tasks. ALL-FEM similarly reports a visually indistinguishable displacement field that was ultimately rejected because code inspection revealed plane strain rather than the required plane stress. MechAgents’ plate-with-a-hole example reports a total reaction force without resolving the 2D thickness convention, illustrating that an apparently precise numerical answer can remain dimensionally under-specified.

The corpus also shows that **retrieval and memory help when they are selective, structured, and traceable**. Foam-Agent 2.0 separates case structure, file details, scripts, and commands into four FAISS indices. MooseAgent retrieves from more than 8,000 annotated MOOSE inputs. PDE-Agents combines a materials/issues knowledge graph with past-run embeddings and finds that a smart, lazy retrieval policy substantially outperforms mandatory retrieval. AgenticSciML retrieves from a curated 70-technique knowledge base, while FeaGPT uses a similarity threshold to reuse prior workflows. The common lesson is not simply “use RAG.” Retrieval must be aligned to workflow stage, constrained by dependency

structure, and followed by deterministic validation. Mandatory or undifferentiated retrieval can introduce latency, distract the model, or fail under tool budgets.

The strongest empirical evidence in the corpus comes from studies that include **repeated runs, controlled ablations, exact reference outputs, or independently constructed verification tests**. Examples include AutoFEA's 512 CalculiX-test corpus and exact .dat comparison; Foam-Agent 2.0's reviewer and hierarchical-RAG ablations over 110 cases; PDE-Agents' 50-case ablation with physics and material-property metrics; FEM-Bench's curated verification inputs plus expected-failure tests; and Tian et al.'s 1,120 randomized runs. Evidence is weaker when results are based on one or a few demonstrations, a small number of hand-selected tasks, a same-model judge, best-of-search selection, or qualitative plot inspection without mesh and discretization control.

The field's most important unresolved problem is therefore **not code generation in isolation**. It is the construction of a trustworthy assurance layer that converts agentic automation into auditable computational science. Such a layer must separate software verification from solution verification and model validation; enforce units and schemas; test manufactured solutions, patch tests, conservation and invariance; quantify discretization and solver error; compare independent implementations or analytical limits; record provenance; and abstain when evidence is insufficient. The next generation of systems should be judged not only by how often they run, but by how often they issue a scientifically defensible claim with a complete evidence package.

Bottom line. The corpus supports the claim that agentic systems can automate substantial portions of CAE, FEM, CFD and SciML workflows. It does **not** yet support unattended, general-purpose, safety-critical simulation. The empirical frontier is reliable orchestration and repair; the scientific frontier is verified, uncertainty-aware, independently auditable reasoning.

Principal findings at a glance

Finding	Evidence across the corpus	What remains unproven	Research implication
Decomposition improves completion	Foam-Agent, Geng, ALL-FEM, MooseAgent, VFEAgent and PDE-Agents show gains from role separation, review loops or structured state.	That more agents intrinsically improve physical correctness. Tian shows redundancy can create premature consensus; ALL-FEM's gain over its best two-agent model is only two medium tasks.	Compare architectures at matched model, token, tool and wall-time budgets; ablate roles individually.
Execution feedback is highly valuable	AutoFEA, MooseAgent, ALL-FEM and Foam-Agent repair syntax, dependency and runtime failures.	That runtime repair detects silent physics errors.	Add physics-triggered diagnostics and independent verification, not just exception-triggered debugging.
Structured retrieval beats generic context	Foam-Agent's hierarchical indices, MooseAgent's input library and PDE-Agents' smart knowledge policy improve success.	Generalization to unseen equations, solvers and version changes without contamination.	Retrieve versioned, authoritative artifacts with dependency graphs and citations; measure retrieval precision and causal utility.

Finding	Evidence across the corpus	What remains unproven	Research implication
Domain adaptation can outperform larger naive models	ALL-FEM's fine-tuned GPT-OSS 120B in an agent loop outperforms a non-agentic GPT-5 Thinking baseline on its 39 tasks.	A model-size conclusion under matched scaffolding and compute; the comparison changes architecture and model simultaneously.	Use factorial evaluations: base model, fine-tuning, tools, retrieval, agents and verification as separate factors.
Current models remain brittle on specialized mechanics	FEM-Bench's geometric-stiffness and critical-load tasks remain unsolved without detailed domain knowledge; Tian's agents fail physical audits.	Whether authoritative retrieval plus symbolic/numerical tools closes the gap robustly.	Target nonlinear kinematics, constitutive integration, contact, dynamics and coupled multiphysics with adversarial variants.
"Success" is not standardized	Metrics range from execution and file presence to exact regression, plot matching, LLM judging, manual review and analytical error.	Cross-paper ranking and deployment readiness.	Establish a verification ladder and report every rung separately.

Review scope, protocol and source integrity

Corpus definition

The review includes every uploaded PDF identified in the chat corpus. Each PDF was extracted and read beyond its abstract and introduction, including methods, algorithms, implementation descriptions, experiments, ablations, appendices, code listings, limitations and conclusions. Claims below are attributed to the uploaded versions; no attempt is made to silently substitute a later online revision. Dates therefore refer to the version visible in each uploaded file, with source-quality qualifications where needed.

The corpus is heterogeneous. Fifteen papers are directly about LLM or multi-agent automation of simulation or scientific coding. Three concern benchmarks or broad field synthesis. One is a doctoral-dissertation preview that summarizes several contributions but omits the substantive dissertation chapters. One is a theoretical paper on an $SO(1,3)$ gauge theory of gravity. Treating all 20 as if they answered the same research question would be methodologically wrong, so the review preserves their distinct evidentiary roles.

Analytical protocol

For each work, the review asks seven questions.

1. What precise problem is addressed, and at what level of the engineering workflow?
2. What mathematical, numerical and agentic model is used?
3. What is the actual code and tool architecture, including state, interfaces, retrieval, execution and repair?
4. What is novel relative to a direct-prompting or single-agent baseline?
5. What data, tasks, metrics, repetitions and ablations support the claims?
6. Which conclusions are warranted by the evidence, and which remain extrapolations?
7. What reproducibility, verification, validation, uncertainty and deployment gaps remain?

The evaluation deliberately distinguishes four levels that are often collapsed in the papers:

- **Artifact validity:** required files or code exist and satisfy schemas.
- **Software execution:** code parses, runs and terminates without an exception.
- **Numerical verification:** the discrete solver implements the intended equations and demonstrates accuracy, stability, conservation or convergence.
- **Physical validation:** the mathematical model and parameters are adequate for the physical use case and uncertainty is characterized.

Source-quality qualifications

The uploaded Guo doctoral dissertation is a **24-page preview**, containing front matter, abstract, table of contents, lists of figures and tables, and Chapter 1, but not the detailed Chapters 2–6. It can support a

map of claimed dissertation contributions, not a full methodological review of the missing chapters. The separate uploaded LLM-CAE paper provides detailed coverage of the dissertation’s agentic TAPS chapter and is reviewed independently.

The Tian manuscript’s filename says 2024, while its internal software versions and references are 2025-era and the PDF metadata indicates creation in November 2025. Because the title page supplies no publication date, this review calls it the uploaded Tian manuscript and avoids using its filename as definitive publication evidence.

The Baker scoping review contains internal numerical tensions: one passage describes 575 records after duplicate removal, whereas another refers to 122 unique records after duplicates; it also describes peer-reviewed eligibility while characterizing a large share of included works as preprints or conference papers. These do not erase the review’s contribution, but they lower confidence in exact selection statistics.

Terminology warning. Li et al.’s **FEA-Bench** expands to “Feature Implementation Benchmark.” It evaluates repository-level Python feature coding, not finite-element analysis. Mohammadzadeh et al.’s **FEM-Bench** is the computational-mechanics benchmark.

Conceptual map of the literature

Five technical families

Family	Papers	Core contribution	Dominant evidence
End-to-end CAE and solver agents	Foam-Agent 2.0; FeaGPT; VFEAgent; MooseAgent; PDE-Agents; Geng et al.; ALL-FEM; AutoFEA; MechAgents	Translate natural-language or visual requirements into solver artifacts, execute, diagnose and post-process.	Case completion, execution, reference-code comparison, ablations and case studies.
Scientific-method and discovery agents	AgenticSciML; ATHENA; LLM-CAE	Evolve models or numerical strategies, retrieve methods, implement them and evaluate scientific outputs.	Best-found numerical error, strategy traces, ablations and selected scientific cases.
Benchmarks and reliability studies	FEM-Bench; EngDesign; FEA-Bench; Tian et al.	Measure code generation, engineering reasoning, self-verification or multi-agent reliability.	Unit tests, rubrics, repeated trials and manual physical audits.
Synthesis and thesis-level framing	Baker et al.; Guo dissertation preview	Map the field or integrate a research program.	Scoping-review charting or high-level dissertation claims.
Non-agentic mechanics/theory boundary cases	AlphaGeo; Wiesendanger	Parameter identification by surrogate-assisted search; classical solution in a gauge theory of gravity.	Numerical inverse examples or analytical derivation.

Recurrent code architecture

Across the agent papers, the modal architecture can be represented as an assurance-oriented pipeline: request normalization; typed intermediate representation; deterministic geometry or mesh tools; solver-specific code or deck generation; sandboxed execution; runtime repair; physics-aware evaluation; provenance and memory. Most papers implement the first six components. Very few implement the last two at the level required for scientific assurance.

Typed intermediate representations are particularly important. FeaGPT uses JSON between planning and downstream CAD/meshing/solver agents. Geng et al. separate natural-language geometry reasoning from an explicit representation that is translated to OpenSeesPy. VFEAgent forms a multimodal scene or model representation before Abaqus code generation. These representations reduce prompt drift, permit schema validation, and make node, element, boundary-condition and material entities inspectable. Their limitation is ontological coverage: a schema proven on frames or a small set of Abaqus drawings can fail when topology, contact, nonlocal constitutive state, load histories or mixed-dimensional coupling exceed the original vocabulary.

Execution layers also converge on a common pattern. A runner invokes the actual scientific software, captures logs, classifies errors, and returns a compact repair target. Foam-Agent extends this to local and Slurm-based HPC execution; VFEAgent isolates modern agent services from Abaqus's legacy Python runtime; MooseAgent iterates MOOSE input cards; ALL-FEM and MechAgents execute FEniCS code; AutoFEA runs CalculiX regression cases. These mechanisms are effective for syntax, missing files, unsupported keywords and some convergence failures. They are intrinsically weak against a wrong but executable formulation.

Evidence hierarchy used in this review

Evidence level	Minimum criterion	Representative papers	Interpretation
E0: demonstration	One or a few selected examples; qualitative success.	MechAgents; parts of FeaGPT and LLM-CAE.	Establishes feasibility, not reliability.
E1: completion benchmark	Multiple tasks with execution or artifact checks.	MooseAgent; VFEAgent; Geng; parts of Foam-Agent.	Measures workflow reliability, not necessarily physics.
E2: reference comparison	Exact outputs, analytical solutions, human-inspected code or discriminative tests.	AutoFEA; ALL-FEM; FEM-Bench; PDE-Agents.	Supports bounded correctness within benchmark scope.
E3: controlled causal evidence	Repeats, role or retrieval ablations, matched comparisons, error taxonomy.	Foam-Agent; PDE-Agents; Tian; portions of FEM-Bench.	Supports claims about why a system helps, subject to confounds.
E4: deployment-grade assurance	Independent V&V, uncertainty, adversarial and out-of-distribution tests, versioned reproducibility, safety case.	None.	Required before safety-critical unattended use.

Detailed review of the 20 uploaded works

1. Adhikari et al. (2026), PDE-Agents

Problem and contribution

PDE-Agents targets automated finite-element simulation of parameterized heat equations rather than open-ended solver synthesis. The system accepts structured problem requirements, configures and runs a DOLFINx-based finite-element workflow, analyzes outputs, and stores experience. Its substantive novelty is the integration of a LangGraph multi-agent orchestrator with a knowledge graph, vector retrieval over prior runs and documentation, and a **selective knowledge policy**. The paper's most informative result is not simply that retrieval helps; it is that always-on retrieval can be slower and less reliable than a policy that invokes knowledge only when the task warrants it.

Architecture and implementation

The supervisor coordinates a simulation agent, an analytics agent, a database or knowledge layer, and deterministic tools. The paper assigns Qwen to simulation-oriented reasoning and Llama to analytics. The infrastructure uses Docker, PostgreSQL, MinIO and Neo4j, with DOLFINx as the numerical backend. Tool calls expose bounded actions rather than unrestricted shell use, and robust JSON parsing is used to recover structured outputs. Supported problem configuration includes geometry, material properties and boundary conditions for the scalar heat equation.

The knowledge layer contains material records, known issues, nine analytical rules, embeddings of previous runs, and chunked documentation with cross-references. Its three policies are: **Off**, which does not use the knowledge graph; **On**, which retrieves it mandatorily; and **Smart**, which uses prompt-level or controller logic to decide when knowledge is necessary. This is an important design distinction because retrieval consumes context, tool calls and time, and can fail independently of the solver.

The workflow is stateful. A request is formalized into simulation parameters, checked against tool schemas, executed through DOLFINx, analyzed, and written back as structured experience. The paper also reports a production-style history of 1,369 runs, with a 97.8% overall success rate and an 85.4% first-try success rate. These operational statistics are useful, although the exact distribution and independence of production requests are not as controlled as the benchmark.

Numerical evidence

Verification uses three heat-equation cases with mesh resolutions from $N = 8$ to 128. Reported error behavior is second order, consistent with the chosen discretization and refinement sequence. Six gallery examples demonstrate supported configurations. The main ablation contains 50 cases spanning easy, medium, hard and novel conditions, and reports execution success together with a physics score and a material-property-fidelity score.

Under the Off policy, success is 100%, the physics score is 0.853, material-property fidelity is 0.796, and average time is 12.9 seconds. Mandatory knowledge reduces raw success to 94% and increases average

time to 60.8 seconds; its overall physics and material scores are 0.846 and 0.752, although success-only values rise to 0.879 and 0.801. The Smart policy restores 100% success, increases the two scores to 0.933 and 0.926, and requires 28.7 seconds. The advantage is most pronounced on novel cases: Off yields physics and material scores of 0.590 and 0.340, mandatory knowledge yields 0.887 and 0.889, and Smart approaches unity for both.

What is genuinely novel

The paper contributes a relatively mature empirical treatment of knowledge use. It shows that knowledge augmentation should be viewed as a **resource-allocation and control problem**, not a binary architectural feature. It also connects simulation provenance to a growing knowledge base, moving beyond static RAG. The separation between solver tools and knowledge tools is appropriate for auditability.

Limitations and research gaps

The numerical scope is narrow: a scalar heat equation with a fixed solver family. The system configures a trusted numerical implementation; it is not shown deriving arbitrary weak forms, generating mixed formulations, selecting stable element pairs, or handling nonlinear constitutive updates. Some material records are intentionally fictitious, useful for testing retrieval but not a validation of real materials science. The mandatory-knowledge failures also show that extra tools create new failure modes. The Smart policy's decision logic needs stress testing against prompt injection, ambiguous material names, stale records and deliberately conflicting sources. Finally, second-order convergence on three cases demonstrates implementation consistency but not broad physical-model validation.

Assessment. Among the uploaded systems, PDE-Agents provides one of the clearest demonstrations that selective, provenance-aware knowledge improves novel-case fidelity. Its claims should be confined to parameterized heat-transfer workflows until expanded to mixed, nonlinear and coupled PDEs with independent solution verification.

2. Baker et al. (2025), Large Language Models in Mechanical Engineering: A Scoping Review

Scope and review method

Baker et al. map LLM applications in mechanical engineering using a PRISMA-ScR-inspired process. Searches cover four databases through June 2025 and target work from 2020 onward. arXiv is explicitly excluded. The reported flow begins with 587 records, removes 12 duplicates, applies an ASReview-assisted screening process, and ultimately includes 66 studies after full-text exclusions. ASReview is seeded with known relevant papers, and screening stops after 50 consecutive irrelevant records. Data are charted into five categories after a five-paper pilot and team calibration meetings.

The review reports that more than 68% of included work appeared in 2024, interprets this as rapid growth, places the United States at nearly half of publications with China second, and finds GPT-family

models in more than 60% of studies. Conceptual design and CAD are prominent application areas. Twenty-eight studies use custom or multi-agent frameworks. Evaluation commonly relies on geometry-related metrics, task success, NLP metrics or qualitative assessment. Weak spatial reasoning and system reliability are recurring limitations, and the review calls for specialist datasets, multimodal systems, common benchmarks and human-centered co-pilot designs.

Contribution

The paper is valuable as a broad field map. It moves beyond individual FEM demonstrations and shows that mechanical-engineering use extends through requirements, ideation, CAD, manufacturing, education and analysis. Its classification of application type, model choice, modality and evaluation exposes how fragmented the evidence base is. The emphasis on spatial reasoning is especially relevant to VFEAgent, FeaGPT and CAD-centered systems in the present corpus.

Methodological concerns

The paper does not report a formal risk-of-bias or quality appraisal of the included studies. A scoping review need not perform meta-analysis, but without even a coarse evidence-quality rubric, a single qualitative demonstration and a repeated controlled benchmark can influence the narrative similarly. Excluding arXiv removes much of the fastest-moving agentic CAE literature and produces a literature map that is systematically more conservative and publication-lagged than the uploaded corpus.

There are internal reporting tensions. The PRISMA numbers describe 575 records after duplicate removal, while Section 2.3 refers to 122 unique records after duplicates. Eligibility is framed around peer-reviewed work, while another passage characterizes a large share of included studies as preprints or conference outputs. The claim of 150% growth also needs an explicit denominator and interval. These issues should be resolved in a corrected flow diagram and machine-readable screening log.

The search vocabulary appears broad but may miss studies using “computational mechanics,” “scientific machine learning,” “PDE solver,” “digital twin,” or solver-specific names without generic mechanical-engineering terms. ASReview can improve efficiency, but seed choice and the 50-irrelevant stopping rule may under-sample conceptually novel subfields. Inter-reviewer agreement statistics are not foregrounded.

Assessment. This is a useful taxonomy and agenda-setting review, but its exact prevalence estimates should be treated cautiously. A stronger update should include preprints in a separately labeled evidence stratum, publish the screened-record dataset and exclusion reasons, correct flow inconsistencies, and score study quality using reproducibility, task breadth, repetition, independent verification and physical validation.

3. Deotale et al. (2026), ALL-FEM

Data construction and fine-tuning

ALL-FEM combines domain-specific fine-tuning with execution-feedback agents for legacy FEniCS. The training corpus contains 1,004 entries: 503 expert seed codes and 501 synthetically generated variants. The synthetic pipeline uses stronger models for problem design and lighter models for code production and repair, followed by expert review. The authors formulate examples in an instruction–response format and fine-tune Llama 3.2 3B, a model described inconsistently as Llama 3.3 7B or 70B, Qwen 3 32B and GPT-OSS 120B. Low-rank adaptation uses rank 8, alpha 16 and dropout 0.15, with reported optimizer, learning-rate, micro-batch, accumulation and hardware settings. Five-fold cross-validation is used for most models, while the largest-model treatment differs.

The supervised objective is token-level next-token prediction. The reported validation curves show decreasing cross-entropy and increasing mean token accuracy with scale. This confirms learning of the corpus distribution but does not itself measure executable or physically correct code; the downstream benchmark is therefore essential.

Two-agent and multi-agent code structure

The two-agent system contains a FEniCS Coder and an Executor. The coder receives the complete problem, writes code, and revises it from runtime errors until execution succeeds or the token budget is exhausted. There is no manual intervention. This is a clean baseline for isolating the effect of the model and fine-tuning.

The richer system uses AutoGen and contains a GPT-OSS Coordinator, Planner, Formulator, fine-tuned GPT-OSS FEniCS Coder, Executor, fine-tuned GPT-OSS Corrector, GPT-OSS Evaluator and a human-facing Admin. The Formulator identifies PDEs, variational forms, boundary conditions, spaces, mesh and requested outputs. The Coordinator dynamically selects speakers rather than enforcing a fully fixed graph. When code execution fails, the coder–executor–corrector loop is invoked. When it succeeds, an Evaluator checks alignment. A human uses the Admin only to terminate after the Evaluator is satisfied because the Coordinator otherwise continues selecting agents indefinitely. This means the system is not fully autonomous, even though the human is not reported to provide technical corrections.

Benchmark and evaluation

The 39-test benchmark contains 13 easy, 13 medium and 13 hard problems: 16 solid, 15 fluid and 8 multiphysics tasks, including linear and nonlinear elasticity, plasticity, Newtonian and non-Newtonian flow, thermofluids, fluid–structure interaction, phase separation and moving-domain transport. Five problems introduce unseen PDEs or couplings.

Evaluation is stronger than an execution-only metric. Non-executable code fails immediately. Executable solutions are plotted with the reference solution and compared on standardized axes and characteristic lines. An LLM judge flags code-level discrepancies, which are then manually verified. This catches silent formulation errors: one solution visually matches the reference but uses plane strain instead of the

required plane stress. A five-problem appendix compares this procedure with relative L2-error thresholds, although the benchmark does not use a uniform L2 criterion because independently generated meshes differ.

Results and interpretation

In the two-agent setting, both base and fine-tuned Llama 3.2 3B solve no tasks. Base Llama 3.3 70B generates executable code for nine solid-mechanics tasks but solves none correctly. Fine-tuned Llama 3.3 70B solves 8 of 39, Qwen 3 32B FT solves 5 of 39, base GPT-OSS 120B solves 23 of 39, and GPT-OSS 120B FT solves 28 of 39, or 71.79%. The latter consists of 13 easy, 10 medium and 5 hard tasks. The multi-agent system solves 30 of 39, or 76.92%, adding two medium tasks while matching the two-agent GPT-OSS FT system on easy and hard cases. The non-agentic GPT-5 Thinking reference solves 27 of 39.

The data support two conclusions: domain adaptation improves every tested model family, and execution-feedback scaffolding with a specialized model can outperform a stronger model used without equivalent tools. They do **not** isolate a pure model-size effect, because model architecture, training, inference behavior and reliability differ. They also do not establish that multi-agent decomposition is broadly superior: against the best matched two-agent system, the gain is two tasks and comes with human termination and much larger context exchange.

The cost appendix reinforces this nuance. On Sample Problem 1, the two-agent system uses 45,776 tokens and six minutes, while the multi-agent system uses 716,436 tokens and eleven minutes. On Sample Problem 2, the totals are 1.90 million tokens and 34 minutes versus 1.75 million and 23 minutes. On Sample Problem 3, they are 3.21 million and 42 minutes versus 2.10 million and 29 minutes. Multi-agent overhead can be enormous on simple tasks but may reduce unproductive debugging on hard tasks.

Limitations

The benchmark uses basic canonical 2D geometries and legacy FEniCS; it deliberately under-represents CAD, tagging and 3D meshing. Mesh choices are not standardized, complicating error norms. Human and LLM-assisted code comparison introduces judgment and possible same-family bias. The corpus count is described as both 1,004 and “over 1,000,” while one passage implies 1,005, and the Llama 3.3 size label alternates between 7B and 70B. The synthetic data pipeline risks style and solution-pattern leakage into the test design unless provenance and deduplication are rigorously documented.

Assessment. ALL-FEM is one of the corpus’s strongest demonstrations of domain fine-tuning plus executable feedback. Its best evidence concerns solver-code formulation within a controlled FEniCS ecosystem. General claims about small-model superiority, autonomy or production reliability require matched-budget baselines, fully deterministic termination, standardized numerical verification and substantially harder 3D/nonlinear benchmarks.

4. Geng et al. (2025), a lightweight multi-agent system for 2D frame structural analysis

System design

Geng et al. narrow the task to 2D frame structures and exploit that constraint aggressively. Five Llama 3.3 70B agents cover problem understanding, geometry, code translation, validation and loads. A decisive design choice is separating **natural-language geometric reasoning** from OpenSeesPy code generation. The geometry agent works under four explicit rules governing intersections, connectivity, nodal creation and member division. It emits a structured representation, after which deterministic utilities deduplicate and renumber nodes and elements. The code translator then creates OpenSeesPy, and OpsVis supports visualization.

This hybrid architecture is better described as LLM-guided symbolic model construction than free-form coding. The LLM interprets an architectural frame, but low-level graph consistency is enforced by code. This directly addresses the high frequency of node and element mistakes in the direct-prompting baseline.

Benchmark and results

The benchmark contains 20 frames ranging from three to five bays, including stylized Notre Dame and Louvre geometries and randomized designs. Supports are fixed, a 50 kN horizontal load is applied to the right, and beams carry 10 kN/m. The primary metric is geometric/model completion, with repeated runs.

A vanilla LLM achieves 0 successful models in 200 attempts. Its errors are categorized as element errors in 61%, node errors in 27.5%, support errors in 7%, and material errors in 4.5%. The multi-agent system exceeds 80% on most tasks, reaches 100% on three-bay cases, averages 88% on five-bay cases, and obtains 70%, 70% and 60% on the three hardest frames. In a three-frame, ten-run ablation, merging geometry and code generation yields 0% whereas the modular architecture yields 100%. Gemini 2.5 Pro averages 37% despite 88.5% executable code, demonstrating again that execution is a poor proxy for geometric fidelity; GPT-4o obtains 0% in the reported comparison.

Runtime ranges from roughly 269 to 949 seconds and scales approximately linearly with structural complexity. A reported manual construction takes 17 minutes 24 seconds for one 3–2–3 configuration, suggesting potential productivity gains, although a single manual timing is not a robust human baseline.

Strengths and limitations

The paper provides unusually clear causal evidence for a specific architectural choice: separate semantic geometry from code and enforce graph invariants deterministically. Its error taxonomy is also actionable. Yet the same specialization limits external validity. Frames are planar, orthogonal and governed by a known member grammar. Supports, loading families and solver technology are constrained. No broad response-level validation is reported for displacements, forces, buckling or nonlinear behavior, and there is no independent mesh or element-formulation question because the systems work with beam elements.

The workflow should be extended to non-orthogonal and curved members, semi-rigid joints, releases, offsets, distributed and moving loads, 3D frames, staged construction, geometric nonlinearity, material plasticity and dynamics. It also needs response checks based on equilibrium, symmetry, rigid-body modes, energy and comparison with analytical substructures. Code and prompts should be released with model/version metadata because the reported 70B system has substantial hardware and deployment implications.

Assessment. This work is a strong example of how domain-specific intermediate representations and deterministic graph operations can transform LLM reliability. It validates a design pattern, not a general autonomous structural-analysis capability.

5. Ghorbani et al. (2024), AlphaGeo

Algorithmic model

AlphaGeo addresses inverse parameter identification for constitutive and finite-element models. The method is described as AlphaZero-inspired, but its mechanics differ materially from canonical AlphaZero. It first generates uniformly sampled parameter–response pairs from a simulator. A neural surrogate maps the current parameter state and discrepancy between simulated and observed outputs to a proposed parameter adjustment. Predicted adjustments across observations are averaged to initialize a Monte Carlo tree search. Candidate actions are random perturbations around that average, controlled by a search range, and nodes are selected with an upper-confidence-bound rule. The reward is the negative adjusted relative error, defined as a sum of absolute prediction–observation deviations normalized by one plus the magnitude of each observation. Search includes selection, expansion, simulation and backpropagation.

There is no learned policy/value network updated by self-play in the AlphaZero sense. The closer description is **surrogate-guided stochastic tree search**. The implementation uses NumPy and pandas, with PyTorch and joblib for learning and parallelization. Node and MCTS classes encapsulate tree state and traversal.

Demonstrations

The polynomial demonstration identifies five coefficients. The neural model uses three inputs, hidden layers of 64 and 32 units, and five outputs, trained for 50 epochs on an 80/20 split. Ten thousand forward simulations create the training data. Search parameters vary the number of actions and MCTS iterations, and the adjusted error decreases as search effort increases.

The soil-water retention case identifies four parameters: air-entry pressure, a distribution parameter, a second shape parameter and a pore-connectivity parameter. Training sets range from 5,000 to 50,000 simulations, and the network uses four hidden layers of 64 units. With 2,000 search actions, the adjusted error decreases from 15.57 at 5,000 simulations to 1.11 at 50,000, while data-generation or training time grows from 4.09 to 30.42 in the reported units.

The elastoplastic Toyoura-sand example is the most substantive constitutive case. A bounding-surface soil model exposes five unknown parameters while critical-state, elastic and cyclic parameters are fixed. A Fortran simulator with integration tolerance 10^{-5} produces 10,000 simulations from 15 triaxial tests, requiring 865 seconds. The network takes ten inputs, uses five hidden layers of 256 units and outputs five parameters; training requires 7.69 seconds. Increasing search actions from 100 to 2,000 reduces adjusted error from 185.53 to 80.02 but increases search time from 79.71 to 1,498.59 seconds. The best value is lower than three cited manual calibrations with errors of 176, 136 and 153, although these comparisons are not necessarily matched in bounds, objectives or computational budget.

The light-weight-deflectometer inverse problem couples unsaturated dynamics in displacement and pressure variables with mortar contact. Six tests and 1,000 time steps at 3×10^{-5} seconds are used to infer a stiffness-like parameter K_0 , with initial void ratio and water content included. One thousand simulations require 3,760 seconds. A network with four inputs and three hidden layers of 64 units predicts one parameter. As actions increase from 10 to 500, the estimate approaches the reference value 150, moving from 137.6 to 148.0, while adjusted error decreases from 1.38 to 0.97 and search time rises from 269 to 10,992 seconds.

Contribution and limitations

The paper's contribution is a reusable wrapper for expensive, black-box constitutive and FE calibration. Averaged surrogate adjustments give the tree search a data-driven direction without requiring simulator gradients. The authors correctly recognize nonuniqueness and the importance of bounds.

Nevertheless, the evidence does not establish superiority over standard inverse methods. There are no matched comparisons with Bayesian optimization, CMA-ES, differential evolution, particle swarm, ensemble Kalman inversion, adjoint methods or gradient-based calibration. Search is stochastic, but repeated-run distributions and confidence intervals are absent. The adjusted relative-error objective does not quantify observation noise, parameter posterior uncertainty or identifiability. User-selected parameter bounds may dominate performance. Costs become very large in the FE case, and surrogate error is not propagated into the final calibration.

The "AlphaZero-inspired" name overstates the algorithmic correspondence because there is no self-play policy/value learning loop. A stronger version should use multi-fidelity or active learning, report parameter and prediction uncertainty, analyze identifiability and sloppiness, compare matched budgets, and release simulator wrappers, seeds and data.

Assessment. AlphaGeo is a promising surrogate-assisted search heuristic for simulator calibration, but it is not yet a statistically characterized or benchmarked alternative to established inverse methods.

6. Guo et al. (2025), large-language-model-empowered next-generation CAE

Scientific and software architecture

This paper combines an LLM agent with the authors' TAPS reduced-order modeling framework. The CAE agent communicates through the Model Context Protocol rather than directly manipulating every

scientific artifact. The described repository separates an MCP server, model configuration, solver or data adapters, individual tool modules and an orchestrator. This modularity is a practical contribution: tensor-model construction, code generation, exact-solution generation, execution and plotting can be exposed as typed tools with narrower privileges than an unrestricted coding agent.

TAPS approximates a high-dimensional space–parameter–time field as a finite sum of separated modes. Spatial, parameter and temporal dependence are represented by products of univariate bases, using the authors' C-HiDeNN construction. A generalized Galerkin projection produces lower-dimensional subproblems with Kronecker structure. The method's computational promise comes from never assembling the nominal full tensor grid. Instead, alternating or fixed-point updates solve mode or subspace systems. The paper uses a one-dimensional heat-equation implementation as an in-context template from which GPT-5 is prompted, with chain-of-thought-style decomposition, to construct new tensorized models.

The verification loop is agentic but not independent. The LLM proposes a manufactured or exact solution; an MCP tool generates the corresponding forcing and boundary data; a convergence script runs the generated model; plots and error histories are returned for inspection. The authors explicitly retain human inspection. In the nonlinear diffusion–reaction example, the LLM initially lacks a required weighted mass matrix and a human supplies the formulation, after which the implementation proceeds. This intervention is scientifically reasonable, but it qualifies the autonomy claim.

Demonstrations and reported scaling

Four main examples are reported. A three-dimensional magnetostatic problem is described as equivalent to roughly one billion degrees of freedom and completes in about eight seconds. A three-dimensional elasticity problem is described as roughly three billion equivalent degrees of freedom and completes in about ten seconds. A nonlinear diffusion–reaction problem uses fixed-point iteration and the manually introduced weighted mass matrix, with a reported billion-scale equivalent system and about 27 seconds of computation.

The most dramatic example uses heterogeneous diffusivity with a $4 \times 4 \times 4$ partition, yielding 64 parameters in addition to space and time. The paper describes a nominal tensor grid of approximately 3×10^{156} degrees of freedom and a TAPS computation of roughly 1,100 seconds. An Abaqus comparison on an 80^3 discretization takes 1,860 seconds for a specific parameter set, whereas TAPS is reported at 601 seconds while representing the parameterized family. The workflow is run on an NVIDIA A6000. Approximate human or agent interaction times of five minutes for mathematical setup, ten for coding and five for verification are also reported.

These numbers illustrate the benefit of separated representations, but “equivalent degrees of freedom” and a Cartesian product's theoretical size are not actual assembled full-order systems. They should not be read as direct strong-scaling comparisons. TAPS performance depends on low-rank structure, basis resolution, nonlinear iterations and separability; those quantities need to be reported alongside nominal dimension.

Novelty and evidential limits

The paper's genuine novelty is the combination of a high-dimensional tensor method with an agent that can construct problem-specific implementations and manufactured checks through a tool protocol. It offers a path beyond automating conventional GUI workflows: the agent can help instantiate a mathematically compressed solver family.

The evidence remains case-study level. There are no repeated model runs, prompt or model ablations, comparisons between agent-generated and expert-generated TAPS code, or systematic analysis of failure probability. The same LLM can influence both solver and verifier, creating correlated-error risk. Human insertion of a missing nonlinear operator is a major dependency. Rank selection, truncation error, alternating-solver stopping criteria, conditioning and error decomposition are not characterized sufficiently for the most ambitious scaling claims. The Abaqus comparison is not a like-for-like accuracy and coverage study.

Assessment. The work is conceptually important because it joins agentic programming with a nonstandard reduced-order representation. It demonstrates feasibility, not general autonomous CAE. Future work needs independent manufactured-solution generation, rank and residual diagnostics, matched-accuracy baselines, repeated code-generation trials and complete provenance for every human intervention.

7. Guo et al. (2025), Toward Engineering AGI: EngDesign

Benchmark design

EngDesign evaluates 101 engineering-design tasks across nine domains with 473 human-authored rubrics. The domain counts are operating systems 8, architecture 5, control 18, mechanics 7, structures 13, digital hardware design 17, analog integrated-circuit design 5, robotics 10 and signal processing 18. Sixty-seven tasks are open-source or openly executable; 34 require closed tools or environments; 23 contain images. The average prompt length is reported as 778.71 tokens.

The benchmark is generated through a multi-stage process. Graduate researchers and domain experts propose 186 tasks, an o4-mini filter and human review remove unsuitable items, and faculty-level experts participate in validation. Each instance contains a task description, rubrics, an execution or evaluation pipeline and a reference. An Instructor/Pydantic-based system standardizes reasoning and configuration. The benchmark therefore measures more than prose answers: some tasks require actual code, design artifacts or simulator interaction.

Models, metrics and results

The paper evaluates a wide set of then-current systems, including GPT-4o, DeepSeek V3 and R1, Gemini 2.0 Flash and 2.5, Claude 3.7 and Claude Thinking, o1, o3, o3-high, o4-mini and o4-mini-high. Three trials are used. Pass rate is the strictest metric, while rubric score captures partial completion. The best reported pass rate is 34.38% for o3, and every model obtains 0% pass rate in analog IC design. The best average rubric score is 49.88 for o4-mini-high, narrowly ahead of o3-high at 49.46.

Robustness is reported through the fraction of tasks solved in all trials relative to any trial. Values around 0.62 for o1, 0.61 for o3 and 0.57 for o4-mini-high show that even high-performing reasoning models are inconsistent. On 71 tasks used for iterative refinement, four models receive as many as ten attempts; o3 approaches a 60% cumulative solve rate. This is useful evidence that test-time search helps, but it changes the computational budget substantially.

The error taxonomy assigns roughly 33% to domain-knowledge errors, 25–36% to coding or verification errors, 10–19% to misuse of prior information, 12–13% to hallucination and 6–9% to computational limitations. GPT-4o chain-of-thought prompting raises open-task pass rate from 12.70% to 14.29% and average score from 37.50 to 42.89, but harms some coding tasks, underscoring that a single prompting policy is not uniformly beneficial.

Strengths and concerns

EngDesign’s principal strength is breadth. It exposes engineering work that standard coding or math benchmarks miss, especially simulator configuration, image-conditioned reasoning, multi-file outputs and domain-specific tool use. The rubric structure makes partial failure visible, and the repeated trials reveal instability.

The benchmark also illustrates how evaluator errors can become scientific conclusions. One highlighted control example argues that dividing the numerator and denominator of a transfer function by the same constant changes the controller gain. Algebraically, such a common scaling does not change the transfer function. This calls the local failure attribution into question and demonstrates the need to audit rubrics and post hoc explanations with the same rigor applied to models. A model-configuration table also appears to associate “Gemini-2.0-Flash” with a gemini-1.5-pro identifier. Notation uses N for both tasks and trials in places, reducing clarity.

Broader threats include domain imbalance, task-author and evaluator coupling, closed-software reproducibility, public-task contamination, small trial counts and the possibility that passing a simulator script does not imply an adequate engineering safety margin. Rubrics rarely quantify robustness, uncertainty or failure envelopes. Best-of-ten improvement should be accompanied by total token, tool and wall-time costs and correction for repeated selection.

Assessment. EngDesign is a valuable broad benchmark showing that engineering design remains far from solved. Its own evaluator inconsistency is also a lesson for the field: benchmark truth must be versioned, independently audited and open to challenge. Future releases should publish adjudication records, evaluator tests and domain-normalized scores and should add uncertainty, constraint and safety rubrics.

8. Guo (2026), AI-empowered Next Generation Computer-Aided Engineering: dissertation preview

What the uploaded file contains

The uploaded document is not the complete dissertation. It is a 24-page preview containing title and approval pages, acknowledgments, abstract, table of contents, lists of figures and tables, and Chapter 1. The substantive chapters on TAPS, LLM-empowered CAE, multilevel variational multiscale methods and interpretable tensor decomposition are not present. Any detailed claims about algorithms in those missing chapters would therefore be reconstruction rather than review.

The preview frames a research program around four components. Chapter 2 develops TAPS for very high-dimensional PDEs. Chapter 3 integrates TAPS with an LLM–MCP agent and substantially overlaps the separate LLM-CAE paper reviewed above. Chapter 4 concerns machine-learning-enhanced variational multiscale modeling. Chapter 5 concerns interpretable tensor decomposition. Chapter 6 outlines conclusions and future work.

The abstract and lists of figures/tables claim large computational demonstrations, including a 701^3 voxel model, approximately 343 million degrees of freedom, solved in under seven minutes on a GPU; signed-distance-function and Kaolin-based geometry handling; Tucker or tensor compression; and applications such as a motor rotor. These statements establish the intended scope but cannot be methodologically audited without the missing chapters, equations, algorithms, convergence studies and experimental details.

Critical use in this review

The preview is useful for understanding how the author connects tensor numerical methods, agentic code construction, learned multiscale closure and interpretable decomposition into a single vision of next-generation CAE. It is not independent evidence for each component. The separate journal or preprint versions should carry the evidentiary burden, and duplicated results must not be counted twice in a meta-level synthesis.

Assessment. The appropriate scholarly conclusion is a source limitation: the uploaded preview supports a thesis map and high-level claims only. A complete review requires the full dissertation or the complete chapter papers. This qualification is itself important because polished front matter can make an incomplete source look like a complete monograph.

9. Hou et al. (2025), AutoFEA

Dataset and task formulation

AutoFEA learns to generate CalculiX finite-element workflows from natural-language descriptions by decomposing official regression tests into steps. The source corpus contains 512 CalculiX test cases and 4,792 extracted steps. GPT-4o generates natural-language descriptions. Three random 90/10 train–test splits are used. The system assumes that structural and mesh blocks longer than ten lines can be

truncated or provided, focusing generation on step-wise analysis definitions rather than unconstrained full geometry construction.

Plan graph and code-generation architecture

The planner retrieves a similar existing plan and represents the solution as a graph whose nodes are steps and whose edges encode shared keywords or semantic relations. A two-layer graph convolutional network combined with Transformer attention predicts links, trained with sampled negative edges. The generated or retrieved plan guides a step-wise code generator. Few-shot examples are selected for each step; the resulting fragments are assembled and passed through a final review stage.

This design treats solver input as a structured program rather than a single sequence. Graph links encode dependencies between materials, elements, boundary conditions, steps and outputs, allowing the generator to preserve ordering and reuse. It is one of the earlier corpus examples of explicit plan induction for simulation input decks.

Evaluation and results

Generated jobs are executed with CalculiX, and output .dat files are compared with reference outputs using relative tolerance 10^{-4} and absolute tolerance 10^{-6} . This is stronger than execution-only evaluation and well suited to regression-test reproduction. Reported success is 90.2% with GPT-4o, 81.6% with GPT-4o mini, 75.2% with Gemini Flash, 83.8% with Gemini 1.5 Pro and 78.8% with Llama 3 8B in the full framework. Variants without planning or review perform worse in the reported tables.

The paper also includes a cantilever example that compares an analytical result of 0.0975 with an FEA result of 0.8738 after plasticity is introduced. This is not a validation mismatch if the analytical value assumes linear elasticity while the FE model includes plastic deformation, but it shows that exact context and quantity definitions matter when presenting “analytical versus FEA” comparisons.

Critical assessment

The benchmark largely measures reproduction of known CalculiX tests whose geometry, mesh or long blocks are available or truncated. It does not establish that the system can formulate new engineering models, select discretizations or validate novel physics. Natural-language descriptions are synthetic, and random step-level or case-level splitting can leak highly similar templates across train and test unless families are grouped. The GNN’s contribution is not isolated through architecture, edge-definition and retrieval ablations. Exact .dat equality is excellent for software regression but can reward copying canonical test patterns rather than reasoning.

The paper should add split strategies by analysis family, element type and source template; compare simple sequence plans, dependency parsers and the GNN at matched retrieval; report plan accuracy separately from code accuracy; and include perturbation tests in which geometry, materials and boundary conditions differ systematically from official cases.

Assessment. AutoFEA offers a credible structured-program approach and a rigorous regression metric. Its 90.2% figure means “reproduces held-out CalculiX-style tests under the provided scaffolding,” not “solves 90.2% of arbitrary finite-element problems.”

10. Jiang et al. (2026), AgenticSciML

Framework and evolutionary search

AgenticSciML automates scientific-machine-learning development using a tree of experiments rather than a single repair loop. The user supplies Problem.md, Requirements.md and Evaluation.md and may add Data_config.json. A multimodal data analyst produces exploratory analysis and a text report while being instructed not to inspect validation data. An evaluator generates evaluate.py and scoring guidelines, which a human approves before the search begins. A root engineer produces an initial implementation without knowledge retrieval or debate; this root is also the single-agent baseline.

Subsequent exploration maintains an evolutionary tree. The current best node is always selected as a parent, while an ensemble of Gemini 2.5 Pro, GPT-5 mini and Grok 4 Fast votes for additional exploratory parents. Each node can have at most ten children. A mutation retriever selects zero or one technique from a curated knowledge base and assembles reports from the parent and relatives. Four-agent proposer–critic debate generates a modification strategy; an engineer edits the code, a debugger repairs failures, a result analyst summarizes outcomes, and the evaluator scores the run.

The 70-entry knowledge base links SciML techniques to papers and GitHub implementations. Claude-assisted structuring is followed by human curation. Role allocation is heterogeneous: Gemini handles proposing, retrieval and analysis; Claude Haiku is used for the root and engineer; GPT-5 mini serves critic and debugger. This model diversity may reduce correlated error, but it also confounds claims about the architecture with claims about model assignment.

Scientific tasks and results

Six problems cover a discontinuous function, an L-shaped-domain Poisson PINN, a Burgers PINN, antiderivative operator learning, a multi-input reaction–diffusion operator and sparse/noisy cylinder-wake reconstruction. The best strategies are tailored rather than generic: domain decomposition or specialized sampling for discontinuity and re-entrant geometry, loss and optimization changes for PINNs, operator-architecture changes for functional mappings, and physics-informed reconstruction for flow.

Reported champion improvements over the root range from about one order of magnitude to more than four orders, with the largest exceeding 11,000-fold on a selected metric. A knowledge-base ablation on one task reports mean-squared error 1.4631×10^{-3} for the full system, 3.4208×10^{-3} without knowledge and 3.0315×10^{-2} with random knowledge, using the same root, configuration and six iterations with batch size four. API cost ranges from \$2.07 to \$11.30. On an A6000, representative GPU times exceed LLM-interaction times: the Poisson task uses about 5.6 GPU hours versus 1.7 LLM hours, and the multi-input task about 10.7 versus 2.1.

Novelty and limitations

The paper advances agentic scientific coding by separating **scientific strategy search** from debugging. It records experiment genealogy and lets agents use papers and implementations as mutation operators. This is closer to automated research engineering than template generation.

Yet the root baseline is a one-call engineer, while the full system receives many models, retrieval, debate, iterative execution and best-of-tree selection. The improvement therefore measures a large test-time-compute package, not a pure multi-agent effect. The knowledge ablation covers one task and apparently one search realization, with no variance. A technique being absent from a 70-item knowledge base does not make a discovered combination scientifically novel in the literature. The human-approved evaluator can inadvertently encode the desired solution, and repeated selection against one evaluation function can overfit the validation contract. API costs omit GPU, human and curation costs.

Future work should pre-register evaluation metrics, keep a sealed test set that is never queried during evolution, compare equal numbers of model calls and GPU hours, repeat complete trees with multiple seeds, perform factorial role and knowledge ablations, and subject claimed discoveries to literature review and independent reproduction.

Assessment. AgenticSciML is an ambitious and technically coherent research-automation framework. It demonstrates that structured experimental search can greatly outperform a weak root implementation on six curated tasks. It does not yet establish autonomous scientific discovery or unbiased generalization beyond the evaluation environment.

11. Li et al. (2025), FEA-Bench: repository-level feature implementation

Scope and construction

FEA-Bench evaluates the implementation of new features in existing Python repositories. Its acronym refers to **feature implementation**, not finite elements. The dataset contains 1,401 task instances from 83 repositories; a Lite subset contains 200 instances from 48 repositories. Instances are derived from pull requests that add components. Each contains a base environment, a natural-language intent, a gold patch and test patch, and metadata about new functions or classes. Filtering requires at least 25% of edited lines to belong to new components, and GPT-4o is used to classify whether a pull request represents a new feature.

Compared with SWE-bench, the tasks are unusually generative. The average oracle-modified code footprint is 2,115.5 lines across 2.62 files, with 128.5 edited lines. New-component lines average 87.1, or 67.8% of edits, and tasks add an average of 4.49 functions and 0.78 classes. This distinguishes feature implementation from localized bug fixing.

Context and inference variants

The benchmark varies the information supplied. **Brief** context gives signatures and concise requirements; **Detailed** context also describes non-Python changes and richer implementation constraints. File context is either oracle modified files or files retrieved from a 27,000-token budget using BM25. Models output a natural-language before/after format or a patch, and the authors also test an Agentless-style workflow on the Lite set.

Oracle-file retrieval is substantially easier than realistic localization. BM25 achieves 40.26% precision and 76.04% recall for target files, but only 51.61% of tasks retrieve all needed files. A 40,000-token budget raises all-file recall to 77.14% yet can lower model performance, demonstrating that more context is not automatically better.

Results

On the full benchmark with oracle files and detailed context, DeepSeek R1 resolves 9.92%, DeepSeek V3 8.14%, o1 7.28%, GPT-4o 6.14%, GPT-4 4.71% and Qwen 32B 4.43%. DeepSeek R1 reaches 14.5% on Lite. Patch-format application rates are much worse than the more forgiving natural before/after representation. In Agentless-style Lite experiments, o1 resolves 14% and GPT-4o 9%.

Complexity sharply reduces success. Tasks adding one function are resolved at 18.96%, two functions at 8.24%, and three or more at 5.47%. This is one of the corpus's clearest measurements of compositional degradation at repository scale.

Relevance to agentic CAE and limitations

Although not a mechanics paper, FEA-Bench is highly relevant to scientific software agents. Mature CAE work often requires modifying a solver repository, adding a constitutive model, preserving APIs and passing regression tests, not merely generating a standalone script. The benchmark shows that repository localization and multi-component consistency are major bottlenecks.

Its scope is limited to Python and single-round implementation. Public pull requests create contamination risk. Test success is an incomplete proxy for maintainability, numerical stability, security and architectural fit. Giving exact signatures constrains the design space, while oracle files bypass a critical real-world problem. Gold tests may reward the original implementation rather than behaviorally equivalent alternatives. Future scientific variants should include C++, Fortran and mixed-language solvers; numerical regression and conservation tests; performance constraints; and hidden tests that perturb mesh, precision and parameter regimes.

Assessment. FEA-Bench demonstrates that current models are weak at repository-scale feature construction even when the relevant files are known. For agentic CAE, it is a warning that impressive notebook generation does not translate automatically to maintainable solver development.

12. Mohammadzadeh et al. (2026), FEM-Bench

Benchmark philosophy and task structure

FEM-Bench is a 33-task diagnostic suite for single-pass generation of computational-mechanics functions and physics-aware pytest tests. It targets four capabilities: domain knowledge, compositional reasoning, algorithmic fidelity and self-verification. Tasks are written as executable Python modules containing a reference function, detailed docstring, allowed imports, optional helper functions, curated verification inputs, reference tests, expected-failure implementations and a `task_info()` metadata function. This **source-first** design makes every instance inspectable and runnable.

Prompts are generated deterministically from templates. Code prompts require exactly the requested function, forbid extra imports and include version and dependency information. Test prompts require pytest-style functions and supply the function under test plus named testing goals. Each model receives an independent API call at temperature 0.1; outputs do not share cross-task memory. The evaluator parses with `ast.parse()`, extracts the first function, rejects unauthorized imports and executes in a controlled namespace.

CC/H/T difficulty system

The CC/H/T identifier is unusually informative. CC0 denotes linear-elastic or basic discretization concepts; CC1 denotes elastic critical-load and geometric-nonlinearity concepts. H is the number of helpers used by the reference implementation. T0 means no helpers are needed or supplied, T1 supplies all helpers, T2 supplies a subset, and T3 withholds helpers that the reference uses. This separates inability to compose known components from inability to reconstruct specialized domain knowledge.

The suite spans FEM 1D, FEM 2D and 3D matrix structural analysis. It covers uniform meshes, bar stiffness, Tri6 and Quad8 shape functions, quadrature, physical gradients, distributed loads, 3D beam transformations, local and global elastic stiffness, geometric stiffness, internal loads, boundary partitioning, linear solves and generalized eigenvalue buckling. The hardest pipeline first solves a linear problem, computes element forces, assembles geometric stiffness and solves $[K_{\text{elastic}} + \lambda K_{\text{geometric}}]\Delta = 0$.

Metrics

Function correctness is binary: all curated inputs must match the reference recursively for scalars, arrays, dictionaries and nested structures within specified tolerances. Runtime failure is incorrect. Generated tests are evaluated in three stages: they must pass the reference implementation, fail every known-incorrect implementation, and therefore achieve joint success. Aggregate metrics include function correctness, reference-pass rate, expected-failure detection and joint success.

Expected failures are not only trivial zero outputs. They include swapped inertias, sign and ordering errors, omitted geometric-stiffness physics, missing conditioning checks and wrong input behavior. Example tests compare Euler buckling loads over radius and length sweeps, test orientation invariance and mode transformation, require mesh convergence, verify cantilever deflection, check equilibrium

and linearity, and test quadrature exactness. This is much closer to scientific self-verification than ordinary unit-test synthesis.

Model results and failure modes

In a single run, Gemini 3 Pro solves 29 of 33, Claude Opus 4.5 solves 28, Gemini 2.5 Pro 26, GPT-5 mini 25, GPT-5 22, Qwen Coder 21, Claude Haiku 19, Qwen Next 16, Llama Maverick 16 and Llama Scout 6. Average joint test-success rates are approximately 71.6%, 63.4%, 71.9%, 49.5%, 73.8%, 65.4%, 41.8%, 49.3%, 41.4% and 37.2% in the table's model order, showing that the best code generator is not automatically the best test generator.

Five-run evaluation of the three leading providers reveals instability. Gemini 3 Pro solves 30 tasks at least once and 26 in all five runs; Claude Opus solves 29 at least once but only 16 in all five; GPT-5 solves 28 at least once and 19 in all five. Assemble-global-geometric-stiffness T3, elastic-critical-load T3 and local-geometric-stiffness T0 are never solved by any model. Providing all helpers makes critical-load T1 broadly solvable; providing the geometric-stiffness helper in T2 enables many flagship successes; removing it causes collapse. The error analysis therefore identifies a concrete missing knowledge object rather than a vague "reasoning" deficit.

GEPA prompt optimization

The appendix optimizes system prompts with GEPA. Thirty tasks are used for optimization and the three critical-load tiers are held out; the local geometric-stiffness task is nevertheless in the training set. A 70/30 train-validation split, minibatch size three and light, medium and heavy budgets are used. Light and medium optimization often produce generic advice with no improvement. Heavy optimization enables Gemini and GPT-5 to solve all held-out tiers only after the evolved prompt incorporates extensive domain-specific formulas, including the full 12×12 geometric-stiffness matrix and force-extraction conventions. This does not show a general prompting breakthrough; it shows that authoritative domain knowledge closes the bottleneck.

Limitations and opportunities

The suite is small and introductory, excludes material nonlinearity, contact, dynamics, incompressibility and coupled fields, and uses single functions rather than full solver repositories. Public release invites future contamination. Reference-output matching can miss defects outside curated inputs, and expected failures define only a finite mutation set. Tiers are not perfectly monotone because task semantics and helper choices vary. The GEPA conclusion is confounded by training on the missing component and by injecting near-solution-level content.

Assessment. FEM-Bench is one of the most scientifically useful papers in the corpus because it operationalizes the difference between writing code and testing physics. Its central result is precise: models can compose known FEM components but remain unreliable when specialized geometric-nonlinearity knowledge must be reconstructed, and their self-generated tests are weaker than their implementations.

13. Ni et al. (2023/2024), MechAgents

Architecture and case studies

MechAgents is an early demonstration of LLM multi-agent collaboration in mechanics. It uses AutoGen pyautogen 0.1.14, GPT-4, Python 3.10 in Google Colab and FEniCS. The two-agent configuration has a user-proxy executor and an assistant that plans, writes, runs and debugs code. The multi-agent configuration adds an admin, planner, scientist, engineer, executor, critic and group-chat manager with dynamic speaker selection.

In the first conversation, the system solves a mechanics problem over five rounds but confuses von Mises stress with the shear component σ_{xy} until a human corrects it. A second conversation progresses from linear elasticity to compressible Neo-Hookean material under 50% prescribed displacement. These traces show real iterative code construction, but they are demonstrations rather than controlled experiments.

The main comparison uses a copper plate with a hole and a prescribed displacement of 0.02 m. The two-agent system makes formulation or code errors; the multi-agent system's planner, scientist and critic identify and correct them. The final reported total reaction is $2.05260850474 \times 10^9$ N and is treated as correct.

Scientific significance and limitations

The paper's novelty is historical and architectural: it demonstrates that role specialization can combine constitutive knowledge, numerical coding, execution and critique in one conversation. The transcripts make failure recovery visible and helped establish the multi-agent mechanics research direction represented by later works.

The evidence is extremely limited. There are no repeated trials, model baselines, ablations, predefined acceptance tests or quantitative response-error metrics. Correctness rests largely on author inspection and the agents' own agreement. The plate example also contains a dimensional ambiguity. A two-dimensional plane-stress or plane-strain model produces force per unit out-of-plane thickness unless thickness is explicitly assigned; reporting an unqualified result in newtons can be misleading. Mesh convergence, hole-boundary stress concentration, equilibrium, energy and analytical comparison are not documented as acceptance gates.

The traces further show that a critic can catch semantic errors, but a human correction is sometimes decisive. Later reliability work by Tian et al. directly challenges any inference that adding critics guarantees physical correctness.

Assessment. MechAgents is a valuable feasibility paper and architectural precursor. It should be cited for demonstrating multi-agent mechanics workflows, not for establishing reliability, numerical accuracy or autonomous discovery.

14. Qi et al. (2025), FeaGPT

End-to-end GMSA pipeline

FeaGPT proposes a geometry–mesh–solver–analysis pipeline with a planning agent, a FreeCAD agent, a Gmsh agent, a CalculiX agent and an analysis agent. Gemini 2.5 Pro is used for planning. An all-MiniLM-L6-v2 embedding model produces 384-dimensional vectors for case retrieval; cosine similarity above 0.85 triggers reuse from a growing knowledge base. A JSON intermediate representation passes geometry, materials, loads and solver requirements between agents.

Geometry is either retrieved or generated as FreeCAD Python using documentation and four example scripts. An AST-based checker applies whitelists and blacklists and performs topology checks before code is executed. Meshing supports four nominal densities, 0.2, 1, 2 and 5 mm, and distance/threshold refinement. Jacobian and topology checks are applied, physical groups are assigned, and first-order tetrahedral or second-order hexahedral elements can be selected. The CalculiX agent generates keyword cards and runs the solver; the analysis agent parses FRD output and computes stress, mass, safety and fatigue indicators.

NACA 4412 campaign

The principal study enumerates 432 NACA 4412 configurations from a $3 \times 6 \times 6 \times 2 \times 2$ design: three material choices, six chord-related settings, six span or thickness-related settings, two mesh choices and two loading or analysis choices as defined in the paper. Geometry generation succeeds for all cases with a mean reported time of 12.4 minutes and a reported geometry-quality statistic near 0.97. Meshes range from 119,627 to 140,578 tetrahedra, with mean 129,390 and coefficient of variation 4.6%; all reported Jacobians are positive and mean meshing time is 34 seconds. CalculiX converges for all cases in 15–20 iterations.

Eight loading cases are used in the performance analysis. Reported maximum stresses range from 273.58 to 1,108.97 MPa, masses from 77.28 to 188.22 g, and safety factors from 0.45 to 1.84. Sensitivity and Pareto analyses relate material and geometric variables to mass and stress, and a fatigue calculation is included. Additional turbomachinery demonstrations illustrate transfer beyond the airfoil family.

Evidence gaps

The campaign demonstrates impressive batch orchestration but leaves important engineering definitions under-specified. The paper does not establish a broad independent benchmark against expert-built FreeCAD, Gmsh and CalculiX models. Solver convergence and positive Jacobians do not verify load transfer, element formulation, mesh independence or stress accuracy. Stress extrema in tetrahedral meshes are particularly sensitive to local refinement and singular boundary conditions. The fatigue model, mean-stress treatment, stress concentration and cycle definition require more detail before life predictions can be trusted.

Timing statements are internally difficult to reconcile: approximately 1.5 hours, 7.2 hours and 4 hours appear in different workflow descriptions. There is no agent or retrieval ablation, no repeated-model

variance, and limited comparison with direct prompting or commercial automation. The 432 cases arise from a structured Cartesian sweep; they do not constitute 432 independent problem formulations.

Assessment. FeaGPT is a notable integration of CAD, meshing and CalculiX with a reusable JSON contract. Its strongest claim is high-throughput execution of a constrained design family. Physical fidelity, fatigue credibility and generalization to arbitrary CAD and boundary topology remain open.

15. Tian et al., Collaboration Dynamics and Reliability Challenges of Multi-Agent LLM Systems in FEA

Experimental design

Tian et al. perform the corpus's most direct reliability study of multi-agent mechanics. Using AutoGen 0.7.5, GPT-3.5 Turbo, Python 3 in Colab, temperature 0, seed 42 and a 2,048-token cap, the authors revisit four MechAgents-style tasks. Seven configurations combine Coder, Executor, Critic, an additional Critic and a Rebuttal agent: Coder–Critic; Coder–Executor; Coder–Executor–Critic; Coder with two critics; Coder–Critic–Rebuttal; Coder–Executor–Critic–additional critic; and Coder–Executor–Critic–Rebuttal.

Each configuration runs 40 times on each of four tasks, for **1,120 runs**. Agent turn order is fixed and randomized runs use a 12-turn limit. For configurations without an executor, generated code is executed post hoc. Evaluation has three levels: execution, specification and visual inspection, and physical audit of constitutive assumptions and parameters for Tasks 3 and 4. Manual criteria and blank or partial-response categories expose failures hidden by aggregate pass rates.

Results

The Executor improves code execution on more complex tasks, but execution can be actively misleading. On Task 2, Coder–Executor produces executable code in every run while achieving 0% task success. On Tasks 3 and 4, blank or partial outputs reach 35% and 50% in some settings, and boundary-condition errors occur in 11.25% of 80 inspected complex-task runs. The Coder–Executor–Critic configuration produces only two visually successful results on each of Tasks 3 and 4, and **none passes the physical audit**.

Rebuttal behavior is particularly revealing. On Task 1 it affirms the preceding assessment in 90.7% of opportunities. Fourteen of sixteen critic diagnoses are wrong, yet 87.5% of those wrong diagnoses are affirmed. Task 2 shows similarly high affirmation rates around 90% and 85%. Occasionally a false criticism triggers a useful correction by chance, but on average the rebuttal agent behaves more like a consensus amplifier than an independent reasoner.

Additional critics do not monotonically improve outcomes. Redundancy can exhaust tokens, repeat an error or create premature consensus. A generated plane-stress constitutive equation mixes incompatible coefficients; because it is executable and visually plausible, only the physical audit exposes it. The paper thereby demonstrates a reliability hierarchy: runtime feedback repairs software, while constitutive correctness requires an independent domain test.

Interpretation and limitations

The study's main contribution is experimental separation of collaboration dynamics from scientific validity. It directly falsifies the assumption that more agents or more critique automatically produce trustworthy mechanics. It also shows that affirmation is not evidence of independent agreement when all agents share a base model and context.

Limitations include only four mostly linear tasks, one model family, a fixed turn limit and fixed conversation ordering. Some configurations differ both in role and opportunity to execute, creating path and budget confounds. Manual physical audits are expert judgments rather than experimental validation, and multiple comparisons are not adjusted. Temperature zero plus a fixed seed is in tension with the observed 40-run variability unless other service-side nondeterminism or prompt state varies; this deserves explicit explanation.

Assessment. This paper supplies the strongest caution in the corpus. It shows that multi-agent consensus and executable code can coexist with universal physical failure on demanding tasks. Any future agentic FEA paper should adopt a similar multi-level audit and report silent physics failures separately from runtime failures.

16. Toscano et al. (2026), ATHENA

Conceptual model: HENA as a contextual-bandit-like research loop

ATHENA implements HENA, a Hierarchical Evolutionary Numerical Algorithms framework for analytical mathematics, scientific computing and physics-informed machine learning. Each iteration selects a scientific action A , implements it into a code state S , observes execution outputs O and assigns a reward R . The authors describe the process through a contextual-bandit analogy. A first-order implementation operator translates a strategy into code, while a second-order policy or meta-reasoner diagnoses results and proposes the next strategy.

The analogy is useful as an organizational model but should not be mistaken for a trained contextual-bandit algorithm. The policy is an LLM-based expert scaffold, not a learned decision rule with established exploration assumptions. The paper's "submartingale-like" reward and square-root regret interpretations are empirical or heuristic: a roughly linear plot of cumulative regret against VT does not establish the conditional-expectation assumptions of a submartingale or a formal regret bound.

Agent teams and code structure

The Conceptualization Group contains a Coordinator and Gatekeeper. The Coordinator checks whether a problem is well posed, distinguishes forward and inverse tasks, and formalizes the request. After user approval, the Gatekeeper creates an ordered multistage pipeline, for example generating high-fidelity scientific-computing data before training a PINN.

The Storage Group contains an Analyzer and Splitter. It labels notebook cells, extracts PDE and method metadata into JSON, splits long scripts at semantic boundaries and incorporates successful final code into a growing library. This supports agentic RAG over complete templates and smaller modules.

Each generative group has a common structure. A two-agent Strategy Team uses proposer–critic planning based on the request, trial history and Advisor report. The Code Retrieval Team includes a Retriever and Validator for foundational templates plus a Collector and Librarian for enhancement modules and dependencies; the Librarian can generate a missing module. The Implementation Team performs a study phase, maps required edits to cell indices through a Planner Parser, refactors one cell at a time, writes JSON patches, and passes the assembled program to an Inspector that checks fidelity to the strategy report. The Debugging Team names the project deterministically after the first run, executes the program and loops crash reports back to implementation. The multimodal Advisor examines solution, error, residual and convergence plots, maps symptoms to a conceptual scaffold, emits a structured report and updates the trial history.

Models are assigned by claimed strengths: Gemini for long-context strategy, planning and multimodal review; GPT and Grok for criticism; Claude for module generation and repair; Grok and a local GPT-OSS 120B for constrained parsing. This heterogeneous routing is sensible but means architecture and model effects are entangled.

Conceptual scaffolds and scientific results

Four major scaffolds encode universal approximation, optimization, physics-informed ML and numerical methods. Reward combines scientific integrity from 0–50, accuracy 0–20, detail 0–15 and optimality 0–15. The Advisor-approved best checkpoint is retained.

Analytical cases include Black–Scholes, viscous Burgers via Cole–Hopf, rotated anisotropic diffusion, a two-dimensional inviscid problem via characteristics and entropy, and a three-dimensional wave solution using spherical reduction. Numerical cases include Cahn–Hilliard with Fourier discretization, ETD RK4 and dealiasing, and Kelvin–Helmholtz and Rayleigh–Taylor instability using Trixi.jl components such as DGSEM, HLLC or HLL fluxes, Hennemann–Gassner shock capturing and adaptive refinement.

Reported PIML relative errors are 2.07×10^{-6} for Allen–Cahn, 7.87×10^{-9} for viscous Burgers with mean-squared error 2.64×10^{-14} , 7.25×10^{-9} for Helmholtz, 8.13×10^{-9} for Poisson, 7.92×10^{-8} for KdV and 5.65×10^{-4} for inviscid Burgers. Human intervention improves the latter to 3.05×10^{-4} . The inviscid-Burgers run spans 22 iterations and 13 strategies, including viscosity continuation, entropy constraints, Fourier representations and later Chebyshev or KAN changes. Human interventions also include quadrature and wavelet guidance.

An AI-velocimetry case reconstructs a lid-driven cavity from data with 5% noise; a PINN infers Reynolds number 525, field errors are about 6%, and a parameterized FEM check is around 1%. Five-run diversity experiments report perfect task completion but approximately 30% code similarity and three distinct entropy strategies. Operator-learning search evaluates 11 architectures and reports a best QRDeepONet variant with KKAN or RBF components at 5.67% error.

Ablation evidence

The appendix compares direct prompting, a multi-agent system without conceptual scaffolding and full ATHENA. On an inviscid Burgers case, direct prompting uses an undealiased Fourier method with oscillations, the no-scaffold baseline uses a diffusive second-order MUSCL finite-volume scheme, and ATHENA retrieves a characteristics solution. On Rayleigh–Taylor, direct prompting fails, the no-scaffold system introduces a perturbation sign error and low-order smearing, while ATHENA uses high-order DGSEM with AMR and hybrid shock capturing. On KdV and inviscid-Burgers PINNs, the no-scaffold Advisor prematurely accepts poor solutions or hallucinates algorithms; ATHENA reaches much smaller errors with fewer parameters.

These cases support the value of explicit scientific scaffolds. They do not fully isolate it: baseline prompts, library access, run budgets and advisor behavior must be matched, and reported best results are selected after iterative search. The reward itself embeds human expert preferences, so reward improvement is partly improvement against a designed rubric.

Critical assessment

ATHENA is the corpus’s most elaborate attempt to automate **scientific strategy**, not merely code repair. Cell-level patching, dependency-aware retrieval, multimodal diagnosis and persistent experiment history are strong software ideas. Yet the paper’s formal language outruns its proof. The policy is not learned as a contextual bandit; regret is not evaluated across a statistically defined task distribution; and submartingale conditions are not tested. Very small errors can depend on reference normalization, selected checkpoints and highly established library components. Comparisons with literature values are not necessarily matched in data, network size, precision or compute.

Code and data are stated to become public upon publication rather than being available in the uploaded version. Human quadrature and wavelet interventions qualify full autonomy. Independent reproduction, full token/GPU cost, all failed trajectories, sealed test evaluation and same-budget baselines are needed.

Assessment. ATHENA is a significant systems and scientific-workflow proposal with impressive cases. Its strongest claim is that structured conceptual scaffolds guide better strategy search than unscaffolded agents on selected problems. Claims of regret guarantees, autonomous novelty or general research-level reliability remain preliminary.

17. Wiesendanger (2024), Schwarzschild solution in an $SO(1,3)$ gauge theory

Place in the corpus

This paper is unrelated to LLMs, FEM automation or agentic CAE. It is nevertheless part of the uploaded corpus and is reviewed on its own theoretical terms. The goal is to show that a proposed renormalizable local Lorentz-gauge theory contains the Schwarzschild vacuum solution in its classical low-energy limit.

Gauge field and action

Local $SO(1,3)$ transformations act on both spacetime-labeled and spin degrees of freedom while coordinates remain fixed under the “inner” gauge transformation. The covariant derivative is written as a partial derivative plus an antisymmetric Lorentz-algebra-valued gauge field B . Reorganization introduces

$$e_{\alpha}^{\lambda}\partial[B] = \eta_{\alpha}^{\lambda}\partial - B_{\alpha}^{\lambda}\{\partial\zeta\} x_{\zeta},$$

which resembles a vierbein but is a composite functional of the dimension-one fundamental field B . This distinction is central to the author’s claim that the quantum field theory can be renormalizable while the classical description acquires a dimension-zero metric-like object.

The general gauge-invariant action contains dimension-zero, dimension-two and dimension-four contributions. The dimension-zero term is cosmological-constant-like. The dimension-two term combines curvature, three torsion contractions and a torsion-divergence term with coefficients α_i and $1/\kappa = 1/(16\pi\Gamma)$. The dimension-four part contains curvature-squared, derivative and torsion terms. Power counting suggests renormalizability; the paper refers to an earlier work for the full counterterm proof.

At energies below the Planck scale, the dimension-two term dominates. For $\alpha_1 = 1$, $\alpha_2 = -1/4$, $\alpha_3 = -1/2$, $\alpha_4 = -1$ and $\alpha_5 = 2$, the author introduces a torsion-free connection $C[B]$ and rewrites the action as the Einstein–Hilbert functional of $e[B]$. Variation leads to the vacuum Einstein equations for the composite metric $g_{\mu\nu} = e_{\alpha\mu} \eta^{\alpha\beta} e_{\beta\nu}$, with the residual constraint $x^{\alpha} e_{\alpha}^{\lambda} = x^{\lambda}$.

Spherical ansatz and Schwarzschild reduction

A time-dependent spherically symmetric inverse-vierbein ansatz begins with five functions A , B , C , D and E . Imposing the composite-field constraint links B to A and E to C , leaving A , C and D . Setting $D = 1$ normalizes the circumference. The corresponding metric contains dt^2 , $dt dr$, dr^2 and the standard r^2 angular part. A new time coordinate with an integrating factor removes the cross term. The metric then takes the generic diagonal spherically symmetric form. Invoking the usual vacuum Einstein equations and Birkhoff reasoning yields $F = 1 - r_S/r = 1/G$ and hence the Schwarzschild line element. A coordinate transformation maps the solution back to a constrained vierbein of the original form.

What is established and what is not

The derivation establishes existence within the **specially chosen, classically dominant truncation**, assuming the earlier equivalence to Einstein–Hilbert form and the availability of the coordinate transformations under the composite constraint. It does not derive post-Schwarzschild corrections from the dimension-four terms, test solar-system observables, analyze matter coupling, or solve the quantum unitarity problem caused by the noncompact $SO(1,3)$ group. Renormalizability of the full theory is cited rather than reproven. Statements that higher-order terms might explain dark matter or that gravitational quanta might relate to dark energy are explicitly speculative.

The work contains no code or numerical model. Its methodological gap is therefore not agentic reliability but theoretical completeness: uniqueness and stability of the constrained solution, classical perturbations, PPN limits, black-hole thermodynamics, coupling to spin, ghost and unitarity structure, and empirically distinguishable predictions.

Assessment. The paper shows that the proposed theory can recover a key GR vacuum solution after a low-energy coefficient choice reduces its action to Einstein–Hilbert form subject to a compatible constraint. It does not demonstrate full classical equivalence in all settings or a complete viable quantum theory.

18. Yue et al. (2025), Foam-Agent 2.0

Six-agent OpenFOAM workflow

Foam-Agent 2.0 is one of the most complete end-to-end solver agents in the corpus. A LangGraph state machine coordinates Architect, Meshing, Input Writer, Runner, Reviewer and Visualization agents. MCP tools provide controlled file, solver, cluster and visualization operations; Pydantic validates state and messages; LangSmith records traces.

The Architect classifies the request by OpenFOAM taxonomy, retrieves relevant examples and constructs a file-dependency directed acyclic graph. The Meshing agent chooses blockMesh or snappyHexMesh, can import external .msh files and dictionaries, and can generate Gmsh Python scripts. The Input Writer creates files in dependency order: system, constant, initial-condition directory, auxiliary scripts and Allrun. The Runner supports local execution and HPC through Slurm, including job submission and monitoring. The Reviewer consumes state and error history and proposes the smallest corrective edits. The Visualization agent produces and executes PyVista or ParaView scripts and can repair them.

Hierarchical retrieval and tool surface

Tutorial cases are indexed into four FAISS stores: overall case structure, detailed file content, scripts and commands. OpenAI text-embedding-3-small produces 1,536-dimensional vectors, and top-k is five. The MCP surface includes case creation and inspection, dictionary and file operations, meshing, solver execution, log reading, Slurm generation and monitoring, and visualization. This is a strong example of separating reasoning from privileged execution.

Benchmark and ablations

The benchmark contains 110 OpenFOAM cases: 10 shallow or geophysical, 10 combustion, 10 multiphase, 10 shock, 20 turbulent, 30 laminar and 20 heat-transfer cases. Claude Sonnet 3.5 reaches 88.2% success; GPT-4o 59.1%; MetaOpenFOAM 55.5% under one reported mode and 17.3% under another; and a recreated OFGPT alternative reaches 37.3% or 45.5% depending on configuration. Because some baselines are reimplemented rather than run from original authors' code, comparability is imperfect.

Reviewer and dependency ablations are informative. Without reviewer and without file dependencies, success is 48.2% at temperature 0 and 45.4% at 0.6. Adding file dependencies raises these to 56.4% and 57.3%. Adding the Reviewer raises success to 86.4% without dependency tracking and 88.2% with it. Thus the Reviewer is the dominant factor, while the dependency graph mainly reduces repair loops and adds a smaller final gain. Hierarchical RAG reaches 57.3% without the Reviewer versus 44.6% for a single-index alternative; with review, the figures are 88.2% versus 84.6%.

Case studies include external airfoil and tandem-wing flows, Gmsh-generated cylinder and two-square geometries, and a one-million-cell lid-driven cavity on Perlmutter decomposed into 32 subdomains. These demonstrate CAD/mesh diversity and HPC routing beyond toy local runs.

Limitations

The primary success criterion is whether the generated case executes successfully and produces expected artifacts. The paper does not report systematic field-error or quantity-of-interest comparison, grid convergence, conservation residuals, turbulence-model adequacy, y -plus targets, time-step independence or uncertainty. A wrong but stable turbulence model can therefore count as success. Reviewer effectiveness is measured causally, but its physics-detection ability is not.

The published Slurm listing appears to contain typographic or extraction artifacts, so executable scripts should be taken from a versioned repository rather than copied from the PDF. OpenFOAM version, tutorial similarity and possible retrieval leakage must be controlled. HPC success needs resource, decomposition, parallel-efficiency and failure-recovery metrics.

Assessment. Foam-Agent 2.0 offers the strongest workflow-engineering evidence in the corpus: broad task coverage, explicit ablations, dependency-aware files, selective repair and HPC execution. Its 88.2% result is a case-execution rate, not a validated-CFD rate. Adding solver-independent verification is the next decisive step.

19. Zhang et al. (2025), MooseAgent

Workflow and retrieval

MooseAgent automates input-file construction for the MOOSE multiphysics framework. A LangGraph graph aligns the natural-language request, generates an input card, executes it asynchronously, analyzes outputs and iterates through correction. The loop is capped at three iterations. A knowledge base of more than 8,000 annotated MOOSE input files and scraped documentation is embedded with BGE-M3 and indexed in FAISS; the three nearest entries are retrieved.

DeepSeek R1 generates the input card, while DeepSeek V3 handles other roles, with temperature 0.01. This division concentrates the stronger reasoning model on solver syntax and physics blocks while using a cheaper model for orchestration and diagnosis.

Evaluation

Nine benchmark cases are run five times each. The full system averages 93% success; seven cases achieve 100%, thermal–mechanical coupling reaches 80% and plasticity 60%. An ablation without RAG averages 76%, and additional variants show the value of retrieval and correction. The complete evaluation consumes 60,777 tokens and is reported to cost less than one yuan, though provider pricing, caching and date are needed for reproducibility.

Error traces show that most recoverable failures are solver messages, missing blocks, invalid parameters or input syntax. The three-iteration loop is sufficient for many of these. The lower plasticity result suggests that history-dependent constitutive specifications and state variables remain harder than linear templates.

Limitations

The evidence consists of 45 runs over nine cases, a small sample for a framework claiming broad MOOSE automation. Success is primarily execution and expected-output presence, not numerical agreement with trusted reference responses. Retrieval from 8,000 examples raises contamination questions if benchmark cases or close variants are in the library. No task-family holdout or similarity analysis is reported. Error correction may simply converge to a familiar input pattern.

The cost excludes knowledge-base construction, hardware and human review. Model and MOOSE versions should be pinned. Future tests should hold out entire application modules, perturb material laws and boundary conditions, compare Exodus outputs quantitatively, and use convergence and conservation checks.

Assessment. MooseAgent demonstrates that solver-specific RAG plus a short execution loop is effective and inexpensive for familiar MOOSE input patterns. It does not yet establish correctness on novel multiphysics or nonlinear constitutive problems.

20. Zhang et al. (2026), VFEEAgent

Two-stage multimodal architecture

VFEEAgent automates Abaqus model construction from engineering drawings. Stage A uses a vision-language multi-agent system to turn a raw drawing into a structured intermediate representation. Stage B, Text2Abaqus, converts that representation into an Abaqus Python script, executes it and validates the resulting model.

Stage A contains Perception, Reasoning, Validation and Orchestrator agents. The system maintains dynamic belief over incomplete entities, pins critical context and enforces monotonic completeness so resolved fields are not casually discarded. Validation is hierarchical: L1 schema validity, L2 stability and degree-of-freedom consistency, L3 physical plausibility and L4 visual agreement. This layered model is a notable attempt to prevent a VLM from converting uncertain visual cues directly into irreversible solver commands.

Stage B uses AST preflight checks, sandboxed execution, short- and long-term memory and retrieval by structural signatures. It permits at most three corrective cycles and defines a fallback path, although the reported experiments use fallback in 0% of cases. The deployment separates Python 3.9 agent services from Abaqus 2022's Python 2.7 environment, a practically important compatibility solution. Experiments run on dual EPYC 9654 processors with 512 GB memory.

Results

The 15-case benchmark evaluates both perception and execution. On Stage A, VFEAgent obtains schema accuracy 0.900, node accuracy 0.815, connectivity F1 0.648, boundary-condition accuracy 0.600 and overall score 0.704, outperforming the reported baselines. These component scores reveal that connectivity and boundary conditions remain substantially weaker than schema conformance.

On Stage B, VFEAgent reports 1.0 across the main code-generation and execution metrics for all 15 cases, whereas baseline VLM or LLM systems are lower. The result demonstrates that once the intermediate representation is acceptable, typed code generation plus preflight and repair can reliably create the tested Abaqus models.

Interpretation and gaps

The paper's genuine novelty is end-to-end **visual-to-solver** automation with explicit uncertainty and staged validation. The dual-runtime architecture and AST checks are practical contributions. The separation of Stage A and B also localizes failure: perception, not Abaqus scripting, is the current bottleneck.

The benchmark is only 15 cases, and 100% Stage-B execution is not a physical validation result. Connectivity F1 of 0.648 and boundary-condition accuracy of 0.600 are too low for unattended model generation; errors in these fields can dominate every downstream response while still producing a valid Abaqus database. The unused fallback is not empirically validated. The system relies on proprietary software and a legacy interpreter, limiting reproducibility. No mesh-convergence, response-error, contact, nonlinear material or load-history benchmark is reported.

Assessment. VFEAgent is an important multimodal systems paper and exposes a crucial fact: code generation may become nearly deterministic once a correct intermediate representation exists, while extracting topology and boundary semantics from drawings remains unsolved. Future work should calibrate perception uncertainty, require human confirmation for high-impact fields, and evaluate response-level consequences of every extraction error.

Cross-paper synthesis

What the field can currently automate

The papers collectively show that natural-language and multimodal systems can already automate many **procedural** components of simulation. Requirements can be normalized into structured fields; familiar geometries can be scripted; meshes can be generated or selected; input decks can be assembled from solver documentation and examples; jobs can be run locally or on clusters; logs can be classified; plots can be created; and failed syntax can be repaired. These are nontrivial productivity gains because conventional CAE involves extensive glue code, file structure and solver-specific knowledge.

The evidence is strongest when the problem family has a stable grammar. Geng et al.'s orthogonal frames, AutoFEA's official CalculiX tests, MooseAgent's MOOSE input library and Foam-Agent's tutorial-derived OpenFOAM cases all benefit from a finite set of entities and dependency patterns. Once an ontology captures nodes, members, dictionaries, analysis steps or application blocks, LLMs can map language to known forms and deterministic code can enforce invariants.

Performance degrades as the workflow moves from grammar to **scientific judgment**. Choosing a plane-stress rather than plane-strain model, deriving a geometric-stiffness matrix, deciding whether a shock solution needs entropy enforcement, selecting stable mixed spaces, validating fatigue life or identifying an ill-posed inverse problem requires knowledge not reducible to file syntax. FEM-Bench, Tian and ATHENA's ablations all expose this boundary.

Architecture comparison

System	Intermediate representation and control	Retrieval or memory	Execution and repair	Scientific assurance actually demonstrated
PDE-Agents	LangGraph state and typed heat-problem parameters	Knowledge graph, prior-run embeddings, cross-referenced documents; Off/On/Smart policy	DOLFINx tools; structured error handling	Three convergence studies; physics and material metrics on 50 cases.
ALL-FEM	Natural-language formulation followed by FEniCS code; dynamic AutoGen chat	Fine-tuned weights rather than explicit RAG	Executor, Corrector and human termination	Plot, line, LLM-code and manual comparison; limited L2 sensitivity.
Geng et al.	Explicit frame graph separated from OpenSeesPy translation	No central RAG	Deterministic deduplication and model execution	Geometry/model correctness; limited response V&V.
AutoFEA	Step graph with GCN-Transformer link prediction	Similar-plan and step examples	CalculiX regression execution and final review	Exact .dat comparison on known-test family.
FeaGPT	JSON contract across planner, CAD, mesh, solver and analysis	384-d embedding, 0.85 reuse threshold	FreeCAD, Gmsh and CalculiX agents	Positive Jacobians and solver convergence; limited independent response validation.
Foam-Agent 2.0	File-dependency DAG in LangGraph	Four stage-specific FAISS indices	Local/HPC runner, minimal-edit reviewer, visualization repair	Broad execution benchmark and ablations; no field/QoI V&V.

System	Intermediate representation and control	Retrieval or memory	Execution and repair	Scientific assurance actually demonstrated
MooseAgent	MOOSE input state with three-iteration loop	More than 8,000 inputs plus documentation, top-3 retrieval	Asynchronous execution and correction	Execution success on nine cases.
VFEAgent	Multimodal IR with belief and monotonic completeness	Structural-signature short/long memory	AST preflight, sandbox, Abaqus repair	Component extraction metrics and execution; no response-level validation.
AgenticSciML	Evolutionary experiment tree	Seventy-technique knowledge base and relational experiment reports	Engineer, debugger and evaluator	Best-found benchmark error; sealed-test and repeated-tree evidence absent.
ATHENA	Strategy report, cell-index plan and JSON patch chain	Dynamic template/module library and trial history	Debug loop plus multimodal Advisor	Strong selected cases and ablation; formal online-learning claims preliminary.

Single agent, two agents or many agents?

The corpus does not support a simple monotone relationship between agent count and quality. Multi-agent decomposition helps when roles correspond to genuinely different information transformations and when handoffs use validated artifacts. Geng’s geometry/code split is causal and dramatic. Foam-Agent’s Reviewer creates a large gain because it consumes execution evidence and performs targeted repair. VFEAgent separates uncertain perception from deterministic solver construction. These are **functional decompositions**.

By contrast, multiplying generic conversational critics can add little independent information. Tian’s shared-model critics often repeat or affirm errors, and an additional critic can worsen token pressure and consensus. ALL-FEM’s multi-agent system improves from 28 to 30 correct tasks relative to its best two-agent baseline, while requiring many more tokens on a simple case and a human stop signal. The correct design principle is therefore not “more agents,” but **orthogonal evidence and constrained responsibility**.

An agent should be added only if it has at least one of the following: a distinct trusted tool; a distinct data source; a distinct mathematical test; a distinct model with demonstrably uncorrelated failure; or authority over a typed artifact that another role cannot silently alter. Otherwise, the role may be theater rather than redundancy.

Retrieval, fine-tuning and external tools

The papers expose three ways of supplying domain knowledge. Fine-tuning internalizes patterns, as in ALL-FEM. Retrieval injects examples and documentation, as in MooseAgent and Foam-Agent.

Deterministic tools compute or validate facts, as in FeaGPT’s CAD/mesh stack and PDE-Agents’ solver interface. Each has a different assurance profile.

Fine-tuning is fast at inference and can improve terminology and formulation, but provenance is opaque and updates require retraining. Retrieval is traceable and versionable but vulnerable to irrelevant context, stale documentation and near-duplicate leakage. Tools provide the strongest guarantees for bounded operations but require careful schemas and cannot decide whether their inputs represent the correct physics. The best architecture combines all three: a domain-adapted model for fluent local reasoning, authoritative retrieval for versioned knowledge, and deterministic numerical tools for calculations and checks.

PDE-Agents provides the clearest evidence that retrieval should be **lazy and conditional**. Foam-Agent shows that retrieval should also be **hierarchical by artifact type**. FEM-Bench’s GEPA appendix shows that generic reasoning prompts do not replace missing domain knowledge; once the full geometric-stiffness formulation is injected, models can compose the larger workflow. These results jointly argue for small, precise, citation-bearing knowledge packets rather than indiscriminate long context.

Benchmarks measure different constructs

Benchmark	Unit of work	Primary metric	Main strength	Main blind spot
Foam-Agent 2.0	Complete OpenFOAM case	Executable case success	110 cases and causal agent/RAG ablations	No systematic flow-field or QoI accuracy.
AutoFEA	CalculiX regression workflow	Exact output match	Strong numerical regression criterion	Reproduction of known test grammar.
ALL-FEM	Generated FEniCS solver	Expert/plot/code correctness	Detects silent plane-stress/strain errors	Mesh-dependent comparison and human judgment.
FEM-Bench	Individual mechanics function and tests	Reference match and joint test success	Discriminative physics-aware self-verification	Small, introductory, public, single-function suite.
EngDesign	Multi-domain engineering task	Pass and rubric score	Breadth and tool realism	Rubric correctness, closed tools and domain imbalance.
FEA-Bench	Repository feature	Hidden tests resolved	Repository-level composition	Python-only; tests omit many quality properties.
Tian reliability study	Multi-agent FEniCS conversation	Execution, task and physical audit	1,120 runs and silent-error analysis	Four tasks and one shared model family.
VFEAgent	Drawing-to-Abaqus model	Extraction and execution metrics	Separates vision from code bottlenecks	Fifteen cases; no response accuracy.

Scores across these benchmarks are not directly comparable. A 90% official-test reproduction rate, an 88.2% executable CFD rate and a 71.79% expert-accepted FEniCS rate do not rank systems on one latent “engineering intelligence” axis. Task priors, provided artifacts, model budgets and correctness definitions differ. Papers should state the **construct** being measured in the metric name itself, for example syntax success, deck execution, reference-regression pass, solution-verification pass or validated-model pass.

Code-structure lessons

Several implementation patterns recur strongly enough to count as emerging best practice.

8. **Typed state before solver code.** JSON, Pydantic or graph schemas should represent geometry, topology, materials, analysis type, boundary conditions, loads, discretization, solver controls and requested quantities. Free-form prose should not directly trigger solver execution.
9. **Dependency-aware generation.** File and module DAGs prevent an agent from writing downstream artifacts before required names, fields or meshes exist. Foam-Agent and ATHENA show the value of dependency structure at different scales.
10. **Deterministic canonicalization.** Node deduplication, element renumbering, unit conversion, schema validation and AST checks belong in code, not prompts. Geng and VFEAgent benefit from this principle.
11. **Minimal, localized repair.** Runtime feedback should identify the failing stage and permit a patch to the smallest artifact. Rewriting an entire case increases regression risk and context load.
12. **Separation of runtime and physics review.** An exception debugger and a scientific verifier must be distinct because their triggers and evidence differ.
13. **Provenance as a first-class output.** Every generated line or solver keyword should trace to the user requirement, an authoritative retrieved source, a deterministic transformation or an explicitly labeled model inference.
14. **Version-pinned execution.** Solver, Python, library, model and prompt versions, random seeds, hardware and tolerances must be recorded. Several corpus comparisons are weakened by ambiguous or inconsistent version labels.

Where the reported novelties are strongest

The most defensible novelties are architectural and evaluative rather than claims of autonomous engineering intelligence.

- PDE-Agents demonstrates conditional knowledge orchestration and quantifies the cost of mandatory retrieval.
- Foam-Agent 2.0 provides a composable, dependency-aware CFD workflow with local/HPC tools and useful ablations.
- Geng et al. show that an explicit geometry IR plus deterministic graph operations removes a major class of frame-model errors.
- FEM-Bench defines discriminative, physics-aware generated-test evaluation using expected failures.
- Tian et al. empirically show that multi-agent consensus is not independent verification.
- VFEAgent separates visual belief formation from solver scripting and identifies perception as the limiting stage.
- ATHENA makes scientific strategy an explicit evolving artifact rather than hiding it inside code revisions.

- AgenticSciML preserves experiment genealogy and treats literature-backed methods as search mutations.

Claims such as “engineering AGI,” “autonomous discovery,” “AlphaZero-inspired” or “contextual-bandit regret” should be read as research framing unless their formal and empirical conditions are independently established.

Research gaps that remain

Gap 1: no shared verification and validation ladder

Most papers use one terminal success label, but engineering simulation requires a hierarchy. A credible benchmark should report at least the following separately.

Level	Required evidence	Typical current status	Needed benchmark mechanism
1. Artifact/schema	Files parse; entities and units are present.	Often measured.	Typed schema tests, unit checks, completeness and provenance.
2. Execution	Solver completes under resource limits.	Common primary metric.	Pinned environments, deterministic logs, crash taxonomy.
3. Code/formulation verification	Intended governing equations and algorithms are implemented.	Sometimes manual; rarely automated.	Weak-form comparison, AST/IR inspection, constitutive and boundary-condition checks.
4. Solution verification	Discretization, iterative and roundoff error are controlled.	Rare.	Mesh/time/order refinement, residual and conservation tests, manufactured solutions.
5. Model validation	Model agrees with physical observations for intended use.	Almost absent.	Experimental datasets, calibration/validation separation, discrepancy models.
6. Predictive uncertainty	Claim includes parameter, numerical and model-form uncertainty.	Absent.	UQ propagation, sensitivity, confidence or credible intervals, decision thresholds.
7. Operational assurance	Failure is detected safely under shift and adversarial inputs.	Absent.	Out-of-distribution suites, abstention, rollback, human sign-off and audit logs.

Without this ladder, execution improvements can be mistakenly presented as advances in scientific correctness. Future papers should publish a vector of pass rates rather than a single score.

Gap 2: silent physics errors are under-instrumented

Runtime logs cannot expose a stable but wrong constitutive law, reversed normal, incorrect load magnitude, missing thickness, inappropriate turbulence closure or nonconverged mesh. The assurance layer must generate **metamorphic and invariance tests** automatically. Examples include:

- doubling a linear load should double displacements and reactions;
- rigid coordinate rotation should transform vector/tensor results without changing invariant quantities;
- zero load should produce zero response after removing prescribed fields;
- global forces and moments should balance reactions;
- energy and virtual work should be consistent;
- conservation laws should close to discretization tolerance;
- a refinement sequence should approach an asymptotic rate;

- limiting parameters should recover a known analytical or simpler model;
- a constitutive update should satisfy objectivity, thermodynamic dissipation and consistent tangent checks.

FEM-Bench begins this program at the unit-function level. The gap is to lift it to full CAE workflows and multiple solvers.

Gap 3: intermediate representations are too narrow and not standardized

Current IRs work well within a solver or geometry family but lack a common ontology. A production-grade engineering IR must represent mixed-dimensional topology, contact pairs, material history variables, coordinate frames, units, load curves, stochastic fields, constraints, solver tolerances, adaptivity, requested observables and evidence requirements. It should distinguish **specified**, **inferred**, **defaulted** and **retrieved** fields and attach calibrated confidence.

Standards such as STEP, IFC, mesh formats and solver schemas can be referenced rather than replaced, but the agent layer needs a solver-independent semantic contract. The research challenge is not only expressiveness; it is round-trip fidelity and formal validation across CAD, mesh and solver adapters.

Gap 4: benchmark leakage and task-family generalization

Retrieval libraries are often built from official examples, and benchmarks often contain examples from the same ecosystem. Random splits can therefore reward template matching. Stronger evaluation should hold out entire PDEs, element families, application modules, geometry grammars, solver versions and documentation eras. Similarity between test tasks and retrieved artifacts should be quantified.

A useful protocol would create four rings: in-distribution variants; compositional recombinations of known components; withheld physics families; and adversarially perturbed tasks with misleading near-neighbor examples. Retrieval should be audited for precision, source authority and causal contribution.

Gap 5: comparisons confound model, agent design and compute

Many papers compare a tool-rich iterative system with a single prompt to a different model. This answers a product-level question—what package works best—but not the scientific question—why. Factorial experiments should vary one factor at a time: base model, domain fine-tuning, retrieval, deterministic tools, execution feedback, critic, planner, verifier and test-time budget.

Token count alone is insufficient. Studies should report model calls, input and output tokens, wall time, solver and GPU time, human minutes, retrieved context, failed executions and monetary cost. Results should include success per fixed budget and a Pareto curve of accuracy against cost.

Gap 6: multi-agent diversity is mostly nominal

Different role prompts applied to the same model do not guarantee independent evidence. Tian's affirmation results make this explicit. Diversity should be measured empirically through error

correlation, not inferred from agent names. Strong ensembles can combine different foundation models, symbolic algebra, dimensional-analysis engines, independent solvers, analytical approximations and expert-authored tests.

The verifier should be epistemically separated from the generator. It should not see the generator's chain of reasoning before constructing its own test, and ideally it should use a different implementation or mathematical representation. Consensus should be weighted by calibration and historical error, not majority vote.

Gap 7: uncertainty, identifiability and model-form error are missing

AlphaGeo exposes parameter nonuniqueness but does not produce posterior uncertainty. FeaGPT performs fatigue and design sweeps without a full uncertainty budget. PDE-Agents retrieves material values without propagating uncertainty. VFEAgent reports perception accuracy without mapping field uncertainty to response uncertainty.

Agentic CAE should track at least four uncertainty sources: requirement ambiguity, parameter uncertainty, numerical error and model-form discrepancy. The system should propagate alternative plausible interpretations, request clarification when response sensitivity is high, and produce decisions conditional on uncertainty rather than one deterministic answer.

Gap 8: meshing is treated as a service rather than a scientific decision

FeaGPT and Foam-Agent automate mesh generation, while ALL-FEM intentionally simplifies geometry. Yet element type, local refinement, boundary-layer resolution, quality, contact discretization and adaptivity are part of the numerical model. Positive Jacobians and successful mesher termination are insufficient.

Benchmarks should include re-entrant corners, thin features, nearly incompressible materials, anisotropy, moving interfaces and multiphysics boundary layers. Agents should justify element order, stabilization and refinement indicators and verify the quantity of interest across a mesh sequence.

Gap 9: nonlinear, history-dependent and coupled physics remain the hardest cases

MooseAgent's plasticity case, FEM-Bench's geometric stiffness, ALL-FEM's hard multiphysics tasks and ATHENA's shock/entropy cases all point to the same boundary. These problems require internal state, consistent linearization, admissibility, stable coupling and path-dependent verification. They are not solved by recalling a weak form once.

Future benchmark suites should prioritize finite strain plasticity, viscoelasticity, damage and fracture, contact, buckling with imperfections, incompressibility, turbulent reacting flow, fluid–structure interaction, thermal–chemical coupling and adaptive transient analysis. Each should include analytical limits, patch or manufactured tests, and known failure mutants.

Gap 10: visual inputs need calibrated semantics and consequence analysis

VFEAgent shows that schema accuracy can be high while connectivity and boundary conditions remain weak. A field-level accuracy does not reveal consequence: one missed support can change the response qualitatively. Visual-to-CAE systems need risk-weighted metrics, calibrated confidence, ambiguity visualization and mandatory confirmation for load paths, constraints, contacts and materials.

Evaluation should perturb drawings systematically—occlusion, low resolution, rotated views, inconsistent dimensions, handwritten annotations and conflicting views—and measure downstream response error, not only extraction accuracy.

Gap 11: reproducibility and source release are uneven

Several works promise future code release, rely on closed models or software, provide code on request, or omit complete prompt/version details. Small typographical and configuration inconsistencies appear in ALL-FEM, EngDesign, Baker and some listings. Reproducibility requires a machine-readable manifest containing source-PDF version, model endpoint and snapshot, system and user prompts, tools, retrieved documents, random seeds, environment images, solver licenses, hardware, task data and evaluation scripts.

Every reported aggregate should link to per-run results, including failures. Best-of-search papers should release the complete search tree, not only champions.

Gap 12: human participation is under-specified

Human involvement ranges from initial request approval to expert code review, evaluator construction, manual formula insertion and termination. These are materially different. A paper should count and categorize every intervention: clarification, knowledge provision, error correction, acceptance, termination and post hoc adjudication. Time and expertise should be reported.

Human-in-the-loop is not a weakness; hidden human labor is. The target should be calibrated allocation of responsibility, with agents handling repeatable work and humans approving high-consequence assumptions.

A research agenda for trustworthy agentic engineering simulation

Proposed reference architecture

A next-generation system should be built around an **evidence graph**, not a chat transcript. The following stages provide a concrete blueprint.

15. **Requirement compiler.** Parse geometry, physics, materials, loads, boundary conditions, quantities of interest, units, tolerances and intended use. Mark every field as supplied, inferred, defaulted or unresolved. Ask targeted questions when ambiguity materially changes the result.
16. **Model-selection layer.** Generate multiple candidate mathematical models, list assumptions and validity regimes, and rank them using dimensionless groups, data availability and intended accuracy.
17. **Typed engineering IR.** Store topology, coordinate systems, fields, constitutive state, coupling, analysis steps and evidence obligations in a versioned schema.
18. **Independent discretization planner.** Select element/volume method, order, stabilization, time integration, nonlinear solver and adaptivity; produce a verification plan before code.
19. **Solver adapters.** Translate the IR deterministically to at least two independent backends where feasible. LLMs may write extensions, but adapters enforce naming, units and schema invariants.
20. **Sandboxed execution controller.** Run resource-bounded jobs locally or on HPC, capture complete logs and prevent unapproved external actions.
21. **Software verifier.** Check AST, keyword validity, dependency completeness, version compatibility and regression tests.
22. **Physics verifier.** Execute manufactured solutions, patch tests, equilibrium, conservation, invariance, limiting-case and refinement checks selected from the model class.
23. **Uncertainty engine.** Propagate parameter, numerical and interpretation uncertainty to the quantity of interest and identify dominant sources.
24. **Independent adjudicator.** Compare evidence without inheriting the generator's rationale. It can accept, reject, request more computation or abstain.
25. **Provenance and audit package.** Emit an immutable bundle containing requirements, IR, sources, code, mesh, logs, results, checks, uncertainties and human approvals.

Benchmark design that would move the field forward

The proposed benchmark should be multi-level and solver-crossing. Each problem family would contain:

- a natural-language and, where relevant, drawing-based specification;
- a canonical semantic IR plus ambiguous variants;
- at least two solver implementations or one solver and one analytical/reduced model;
- hidden perturbations of units, signs, topology and parameters;
- manufactured or benchmark solutions where possible;
- mesh and time-refinement sequences;

- expected-failure mutants spanning software and physics;
- uncertainty-aware acceptance thresholds for quantities of interest;
- sealed test cases from withheld physics and geometry families;
- fixed compute budgets and complete cost accounting.

Scores should be a vector: requirement completeness, IR fidelity, execution, formulation verification, solution verification, validation, uncertainty calibration, abstention quality and cost. A deployment readiness score should require all critical rungs rather than average away a catastrophic failure.

Priority experiments

Priority	Experiment	Why it matters	Minimum credible design
1	Runtime critic versus physics verifier	Tests the corpus's central silent-error problem.	Same generator and tasks; independent verifier; hidden constitutive and BC mutants; repeated trials.
2	Matched-budget agent ablation	Separates architecture from compute.	Same base model and token/tool budget; add planner, RAG, executor, critic and verifier factorially.
3	Cross-solver replication	Detects correlated implementation errors.	Generate equivalent models in two backends; compare invariants and QoIs after mesh refinement.
4	OOD family holdout	Measures genuine transfer beyond templates.	Hold out element, PDE, geometry and solver-version families; audit retrieval similarity.
5	Consequence-weighted visual extraction	Connects VLM metrics to mechanics risk.	Perturb drawings; propagate extraction errors to response; require calibrated uncertainty.
6	Nonlinear history benchmark	Targets the observed frontier.	Plasticity/contact/transient coupling with path tests, consistent tangent checks and energy balance.
7	Full lifecycle cost study	Establishes practical value.	Compare expert, co-pilot and agent teams on time, errors, compute, review burden and maintainability.

Recommended reporting standard

Every future agentic CAE paper should provide the following minimum information.

- Corpus provenance, deduplication and train/test similarity.
- Model names, snapshots, temperatures, seeds, context limits and role prompts.
- Complete tool schemas and privileges.
- IR or state-machine schema and termination logic.
- Solver, library, operating-system and hardware versions.
- All success definitions, tolerances and adjudication rules.
- Per-stage and end-to-end metrics with confidence intervals.

- Repeated runs and full failure taxonomy.
- Matched-budget baselines and role/retrieval ablations.
- Numerical verification and, where claimed, physical validation.
- Human interventions and total human time.
- LLM, CPU/GPU, solver and monetary cost.
- Release of all generated artifacts and failed trajectories subject to licensing.

Integrated conclusion

The 20 uploaded works document a field moving rapidly from conversational demonstrations to executable, stateful scientific systems. The progress is real. Domain-specific intermediate representations reduce geometric and topological errors. Solver execution loops repair large classes of coding failures. Retrieval and fine-tuning inject specialist knowledge. Hierarchical agents can search scientific strategies, and multimodal pipelines can connect drawings to commercial CAE. Several papers now provide meaningful ablations and repeated experiments rather than anecdotes.

The same corpus also shows why autonomy claims must remain narrow. Executable code can implement the wrong physics; multiple critics can share one misconception; a visually matching field can use the wrong constitutive assumption; a benchmark rubric can itself contain an error; and a theoretical dimension count can be mistaken for actual computation. Current systems are therefore best viewed as **high-leverage research and engineering co-pilots** whose strongest capabilities are orchestration, translation, retrieval and repair.

The most important improvement is to make assurance coequal with generation. Every agent-produced result should be accompanied by a machine-checkable evidence graph connecting requirements to models, discretizations, code, execution, verification tests, uncertainty and approval. Agents should be rewarded for identifying ambiguity and abstaining, not only for producing a plot. Independent mathematical tests and cross-solver comparisons should replace conversational consensus as the basis of trust.

If the field adopts that shift, the next generation of research can move from “Can an LLM run this solver?” to the scientifically meaningful question: **Under what assumptions, with what numerical error and uncertainty, and supported by which independent evidence, may this computed result be used for an engineering decision?** None of the uploaded papers fully answers that question. Together, they provide many of the components needed to do so.

Appendix A. Corpus register

No.	Work	Primary topic	Evidentiary role
1	Adhikari et al. (2026), PDE-Agents	KG-augmented heat-equation FEM orchestration	Controlled retrieval-policy and convergence evidence.
2	Baker et al. (2025)	Mechanical-engineering LLM scoping review	Field taxonomy and trend synthesis.
3	Deotale et al. (2026), ALL-FEM	Fine-tuned FEniCS agents	Domain adaptation, execution feedback and expert evaluation.
4	Geng et al. (2025)	2D frame OpenSeesPy agents	Explicit geometry IR and strong modular ablation.
5	Ghorbani et al. (2024), AlphaGeo	Surrogate-guided inverse calibration	Non-LLM optimization boundary case.
6	Guo et al. (2025), LLM-CAE	MCP-driven tensor CAE	Agent-created TAPS models and case studies.
7	Guo et al. (2025), EngDesign	Engineering-design benchmark	Broad tool/rubric evaluation and failure taxonomy.
8	Guo (2026) dissertation preview	Integrated AI-CAE thesis program	High-level map; full methods unavailable in upload.
9	Hou et al. (2025), AutoFEA	CalculiX generation with a plan graph	Exact regression-output evaluation.
10	Jiang et al. (2026), AgenticSciML	Evolutionary SciML agents	Research-strategy search and knowledge mutations.
11	Li et al. (2025), FEA-Bench	Repository feature implementation	Software-engineering boundary benchmark.
12	Mohammadzadeh et al. (2026), FEM-Bench	Mechanics code and test generation	Physics-aware diagnostic benchmark.
13	Ni et al. (2023/2024), MechAgents	Early FEniCS multi-agent demonstrations	Foundational feasibility evidence.
14	Qi et al. (2025), FeaGPT	FreeCAD–Gmsh–CalculiX pipeline	End-to-end design-sweep automation.
15	Tian et al., reliability manuscript	Multi-agent FEA collaboration	Repeated silent-error and consensus evidence.
16	Toscano et al. (2026), ATHENA	Hierarchical numerical/SciML research agents	Strategy scaffolding and best-found scientific solutions.
17	Wiesendanger (2024)	SO(1,3) gauge gravity	Theoretical non-agentic boundary case.
18	Yue et al. (2025), Foam-Agent 2.0	End-to-end OpenFOAM agent	Broad execution benchmark, RAG/reviewer ablations and HPC.
19	Zhang et al. (2025), MooseAgent	MOOSE input automation	Solver-specific RAG and repair.
20	Zhang et al. (2026), VFEEAgent	Drawing-to-Abaqus automation	Multimodal IR and staged validation.

Appendix B. Quantitative results index

Paper	Task scale	Headline result	Essential qualification
-------	------------	-----------------	-------------------------

Paper	Task scale	Headline result	Essential qualification
PDE-Agents	50-case ablation; 1,369 operational runs	Smart KG: 100% success, physics 0.933, material 0.926	Scalar heat-equation configuration.
ALL-FEM	39 tasks	GPT-OSS FT two-agent 71.79%; multi-agent 76.92%	Multi-agent adds 2 medium tasks; human termination.
Geng	20 frames; repeated attempts	Most above 80%; five-bay average 88%; direct baseline 0/200	Narrow 2D frame grammar; geometry-oriented metric.
AutoFEA	512 tests, 4,792 steps	GPT-4o 90.2%	Official CalculiX regression family with provided scaffolding.
AgenticSciML	6 tasks	Up to more than 11,000-fold metric improvement	Large search budget versus one-call root; best-of-tree.
FEA-Bench	1,401 repository tasks	Best full oracle/detailed 9.92%	Feature implementation, not finite elements.
FEM-Bench	33 tasks	Gemini 3 Pro 29/33 single; 30/33 any of five, 26/33 all five	Hard geometric-stiffness tasks remain unsolved.
FeaGPT	432 NACA configurations	100% reported geometry and solver completion	Structured sweep; limited independent stress/mesh validation.
Tian	1,120 runs	No physically valid solution on Tasks 3–4	Strong evidence against execution/consensus proxy.
ATHENA	Analytical, numerical and PIML cases	PIML errors from 10^{-4} to 10^{-9} range	Selected best trajectories; heterogeneous comparisons and human interventions.
Foam-Agent 2.0	110 CFD cases	Claude Sonnet 3.5 at 88.2%	Executable-case success, not field/QoI validation.
MooseAgent	9 cases × 5	93% average	Small, familiar solver-specific set; execution-focused.
VFEAgent	15 cases	Stage B reports 100%; Stage A overall 0.704	Connectivity 0.648 and BC 0.600 remain critical.

Appendix C. Reproducibility and interpretation checklist

- Confirm that the uploaded version, not a later web version, is being cited.
- Distinguish source-reported numbers from values recomputed from figures.
- Do not combine repeated or overlapping dissertation and paper results as independent evidence.
- Treat commercial-solver execution as necessary but not sufficient for scientific correctness.
- Record whether a baseline uses the same model, tools, context, retries and compute.
- Identify whether retrieval contains exact or near-duplicate benchmark cases.
- Separate human technical correction from administrative termination or post hoc scoring.
- Audit formulas and rubrics; benchmark authors and LLM judges can be wrong.
- Require units, coordinate frames and 2D thickness conventions in reported quantities.
- Require mesh, time-step and solver-tolerance studies before interpreting stress extrema or transient outputs.

- Treat “theoretical” or “equivalent” degrees of freedom separately from actually assembled and solved unknowns.
- Interpret formal analogies such as AlphaZero or contextual bandits only to the degree their algorithmic assumptions are implemented and tested.

Appendix D. Suggested vocabulary for future papers

To reduce ambiguity, future work should reserve the following terms for specific evidence.

Term	Recommended meaning
Executable	The code or deck runs to completion under a specified environment.
Solver-converged	The solver satisfies stated algebraic/nonlinear stopping criteria.
Numerically verified	Discretization and iterative error have been assessed against analytical, manufactured or refined reference evidence.
Physically validated	Model predictions agree with independent observations for a defined intended use.
Autonomous	No unreported human technical decisions occur between specification and accepted result; permitted approvals are enumerated.
Novel	The method or result survives an explicit prior-art search and independent expert review, not merely absence from the agent’s retrieval library.
Reliable	Performance is repeated over a defined task distribution with calibrated failure detection and confidence intervals.
Agent	A component with state, goals and tool or artifact responsibility; a renamed prompt alone is not evidence of an independent agent.